影像處理 Image Processing

大學智慧機器人學程 | 高中 AI 機器人課程 | 台灣自編大學教科書

編者：李世淵(機器人叫獸)|助教：李天宇、李宇晴

第 14 章 機器學習基礎

本章學習目標

- 說明機器學習的定義與典範轉移，區分監督式、非監督式與強化學習。
- 理解影像分類的完整流程：資料、特徵、模型、訓練、評估。
- 認識重要的影像資料集(MNIST、CIFAR、COCO、ImageNet/ILSVRC)及其歷史角色。
- 操作 kNN 與 SVM 兩種經典分類器，理解其原理與適用場合。
- 正確切分訓練/驗證/測試集，以準確率、混淆矩陣、精確率與召回率評估模型。
- 診斷過度擬合與擬合不足，並理解偏差-變異權衡在機器學習中的核心地位。

叫獸開講|MNIST：讀懂支票的神經網路與七萬個手寫數字

1980 年代末，一位名叫 Yann LeCun 的年輕研究者來到美國紐澤西州的貝爾實驗室——那個誕生過電晶體、資訊理論、C 語言、Unix 的傳奇之地。他手上的任務很實際：讓機器學會辨認信封上手寫的郵遞區號，好讓美國郵政的信件自動分揀。手寫數字千變萬化，每個人的「7」都不一樣，傳統那套「人工設計規則」的方法在這裡撞牆了——你永遠寫不完「什麼樣算是 7」的規則。

LeCun 走了一條當時還很非主流的路：與其教電腦規則，不如給它大量範例，讓它自己「學」。1989 年，他和同事打造出第一個以反向傳播訓練的卷積神經網路 LeNet，成功辨識美國郵政的手寫郵遞區號。這個系統不斷改良，到 1990 年代初已經進化到能讀取銀行支票上的金額。這不是實驗室的玩具：搭載這套技術的支票辨識系統於 1990 年代初商業化部署，進入美國與歐洲的自動櫃員機;到了

1990 年代末，全美國約有百分之十以上的支票是由 LeCun 團隊的系統自動判讀的。一個學術演算法，悄悄地每天處理著數以百萬計的真實金錢交易。

但要訓練與比較這些「會學習」的系統，需要一套公平的考卷。LeCun 與同事 Corinna Cortes、Chris Burges 於是整理出了 MNIST 資料集：七萬張手寫數字影像(六萬張用於訓練、一萬張用於測試)，每張都是 28×28 像素的灰階圖，標好了 0 到 9 的正確答案。它的來源有個有趣的細節：MNIST 混合自美國國家標準局的兩個資料庫——一個由人口普查局員工書寫(字跡工整、較容易)，一個由高中生書寫(字跡潦草、較困難)，刻意混合是為了讓訓練與測試的難度一致、結論才可靠。這個看似瑣碎的設計考量，其實蘊含了本章要教的重要觀念：資料的品質與切分方式，往往比演算法本身更決定成敗。

MNIST 從此成為機器學習界的「Hello World」——幾乎每一個學習視覺辨識的人，第一個訓練的模型都是 MNIST 手寫數字分類器，您在課程中的跨框架 MNIST 實作(從 TensorFlow 到 Keras 到 PyTorch)正是這個傳統的延續。它也是無數新演算法的第一塊試金石。一個為了解決郵政與銀行實際問題而生的資料集，意外地推動了整個領域三十多年——這印證了一句在機器學習界流傳的名言：「資料，才是驅動這一切的燃料。」

思考題：LeCun 沒有去寫「什麼樣算是數字 7」的規則，而是給機器看幾萬個 7 的範例讓它自己學。這個「從範例學習」而非「被明確告知規則」的思路，與你過去學程式的方式(工程師寫死每一條 if-else)有什麼根本不同?這個轉變為什麼被稱為「典範轉移」?

14.1 什麼是機器學習

14.1.1 典範轉移：從寫規則到給範例

回顧本書前十三章，我們解決問題的方式有一個共同點：人類專家分析問題、設計演算法。想去雜訊，我們設計濾波器;想找邊緣，我們設計 Canny;想描述特徵，我們

設計 SIFT。這是「傳統程式設計」的典範——工程師明確地把「怎麼做」一步一步寫給電腦。

機器學習代表了一次根本的典範轉移(正是叫獸開講思考題的核心)。它不再由人類寫死規則，而是：給電腦大量的「輸入-輸出」範例，讓它自己從中歸納出規律。用一句話對比：傳統程式是「人給規則，電腦套用到資料上得出答案」;機器學習是「人給資料和答案，電腦自己歸納出規則」。這個轉變之所以重要，是因為有太多問題——例如辨認手寫數字、辨識人臉、理解語音——其規則複雜到人類根本寫不出來，但範例卻俯拾皆是。既然寫不出規則，那就讓機器從範例中自己學。

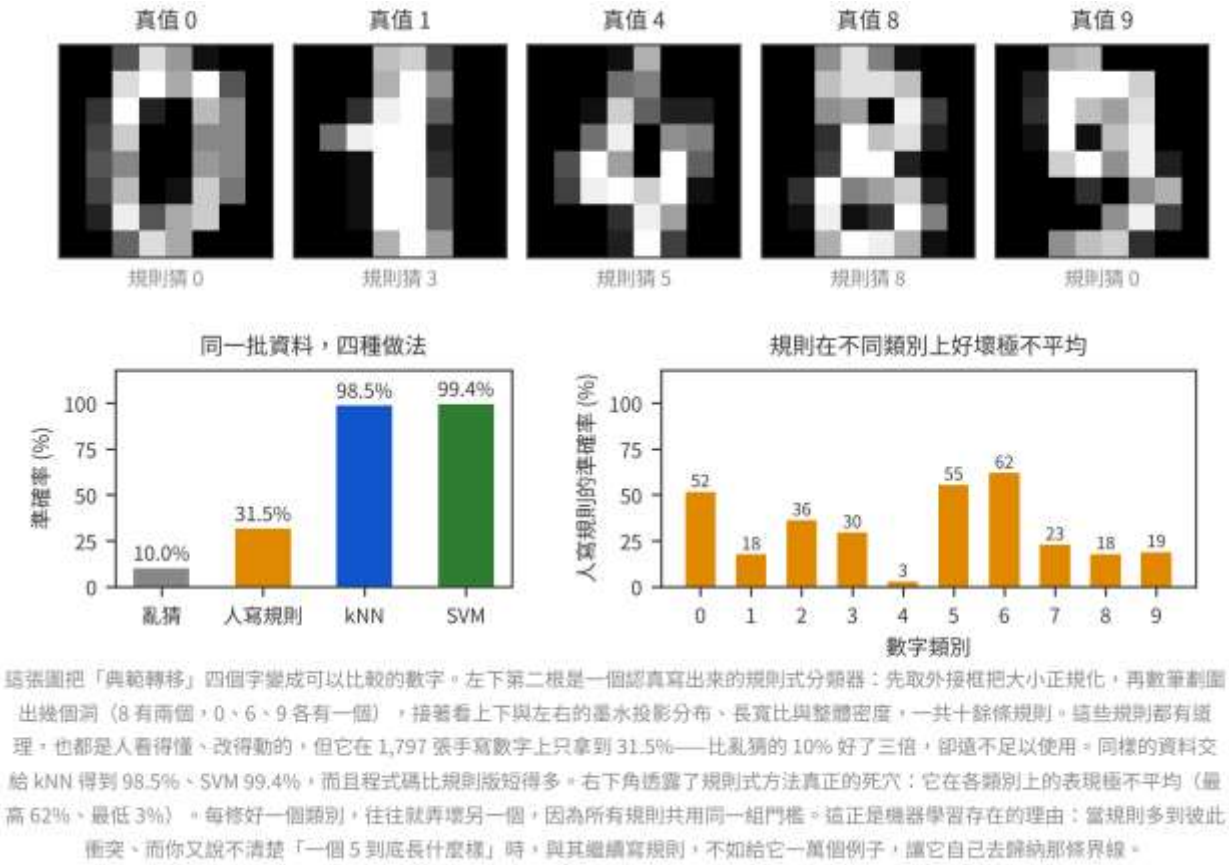


圖 14-1　寫規則與給範例：同一個問題，兩種做法

學術上常引用的定義來自 Tom Mitchell：「若一個電腦程式在某任務 T 上、以某效能指標 P 衡量的表現，能隨著經驗 E 而提升，則稱它從經驗 E 中學習。」以 MNIST 為例：任務 T 是辨認數字、效能 P 是辨識準確率、經驗 E 是那六萬張標好答案的訓練影像。看得愈多、學得愈好——這就是「學習」。

14.1.2 三種學習範式

1. 監督式學習(Supervised Learning)：訓練資料同時提供「輸入」與「正確答案(標籤)」，模型學習從輸入預測答案。本章與 MNIST 都屬此類——每張數字影像都附有正確的數字標籤。它又分為分類(輸出是類別，如「這是 3」)與迴歸(輸出是連續值，如「房價是多少」)。影像辨識絕大多數是分類問題。
2. 非監督式學習(Unsupervised Learning)：訓練資料只有輸入、沒有標籤，模型自己發現資料中的結構。例如分群(把相似的影像自動歸為一類，而沒人告訴它類別是什麼)、降維(把高維資料壓縮到低維以便視覺化或去除冗餘)。
3. 強化學習(Reinforcement Learning)：模型(稱為智能體 agent)透過與環境互動、依據行動的獎懲來學習策略，沒有現成的正確答案，只有「做得好不好」的回饋。它是機器人控制、遊戲 AI(如下圍棋的 AlphaGo)、自駕決策的核心，與機器人學程的關聯尤深——不過本章聚焦影像分類，強化學習留待進階課程。

本章與第 15、16 章的深度學習，主軸都是監督式學習中的影像分類與偵測。理解監督式學習的完整流程，是踏入深度學習的必要基礎——事實上，深度學習只是機器學習的一個分支(用「深層神經網路」作為模型)，它遵循的訓練、評估、過度擬合診斷等原則，與本章介紹的完全一致。先打好本章的地基，第 15 章的 CNN 才不會是空中樓閣。

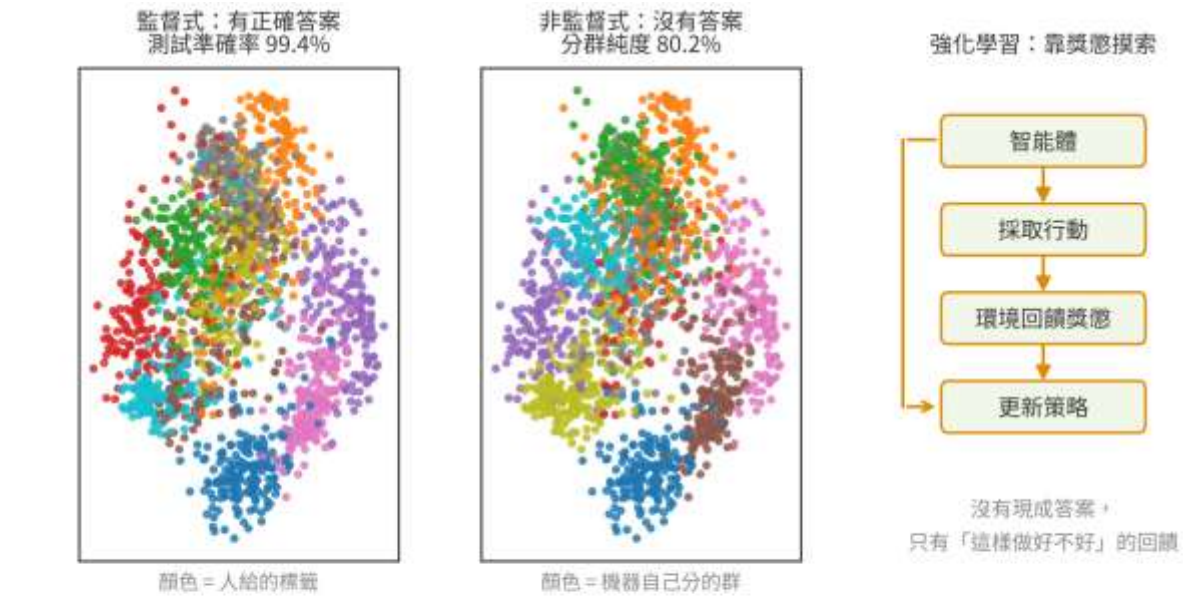


圖 14-2　三種學習範式：有標籤、沒標籤、靠獎懲

14.2 影像分類的完整流程

一個監督式影像分類系統，從資料到可用模型，要走過五個環節。這條流程貫穿本章與後續深度學習各章，務必牢記：

1. 資料(Data)：收集大量影像並標註正確答案。這是整個流程的地基——正如叫獸開講所示，資料的品質與數量往往比演算法更決定成敗。14.3 節專門討論資料集。
2. 特徵(Features)：把原始像素轉換成更有代表性的數值向量。傳統機器學習高度依賴人工設計特徵(第 13 章的 SIFT、HOG、顏色直方圖等)——這一步的好壞直接決定成敗。而深度學習(第 15 章)最大的突破，正是讓模型自動學出特徵，免去了人工設計的負擔。
3. 模型(Model)：選擇一個能從特徵映射到類別的數學模型，如本章的 kNN、SVM，或第 15 章的神經網路。

4. 訓練(Training)：用訓練資料調整模型的參數，使它在訓練資料上的預測盡量正確。
5. 評估(Evaluation)：用「模型沒看過的」測試資料檢驗它的真實表現——這是判斷模型好壞的關鍵，14.5 節詳述。

請特別注意「特徵」這一環，它是理解傳統機器學習與深度學習分野的鑰匙。在傳統流程中，特徵是人工設計的，模型只負責「從特徵到類別」這後半段;而深度學習把「從像素到特徵」與「從特徵到類別」整合進同一個網路，端到端一起學習——這就是第 13.8 節說的「從手工特徵到學習特徵」的典範轉移。本章聚焦傳統流程，正是為了讓你清楚看見：深度學習到底改變了哪一環、又保留了哪一環。

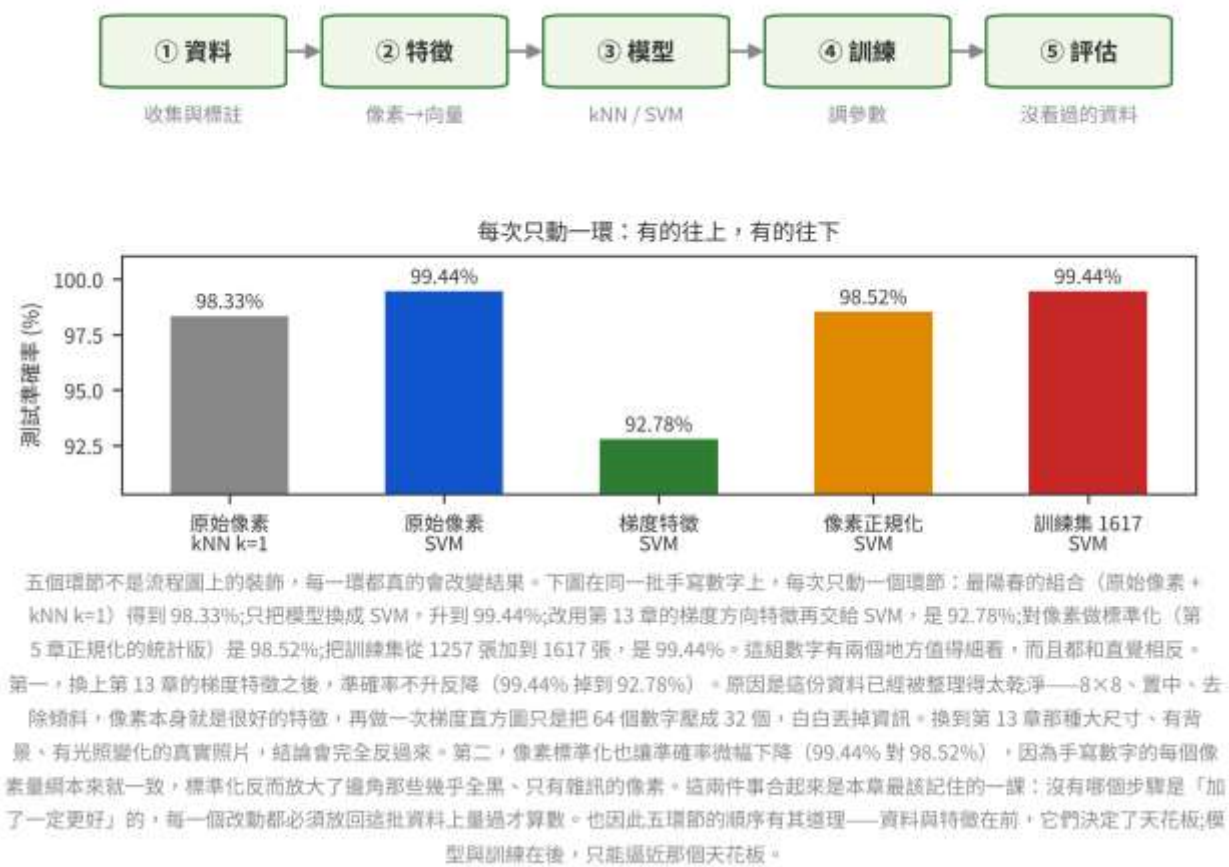
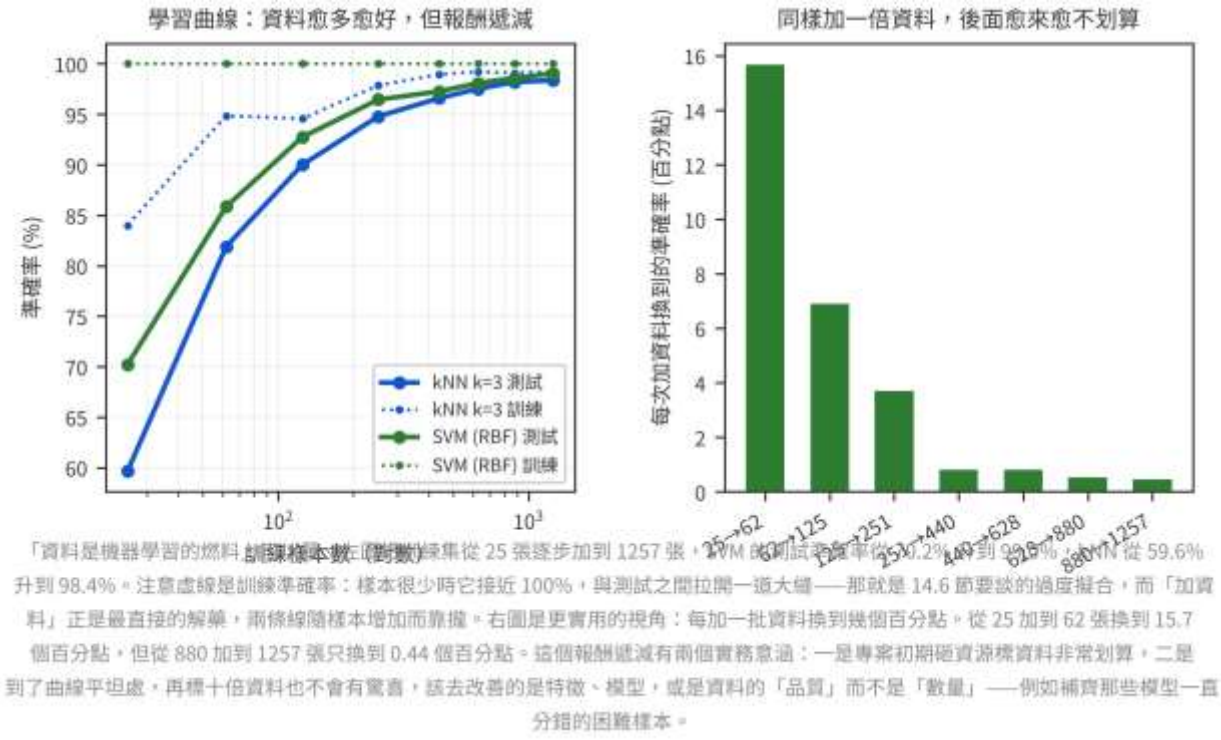


圖 14-3　五個環節：每一環各值多少分

14.3 影像資料集：機器學習的燃料

如果說演算法是引擎，那麼資料就是燃料——沒有燃料，再好的引擎也跑不動。

機器學習的發展史，某種程度上就是一部「資料集推動演算法」的歷史。以下四個資料集，不僅是實作練習的素材，更是理解整個領域演進的關鍵座標。



「資料是機器學習的燃料」這句話在這張圖裡看得一清二楚。訓練集從 25 張逐步加到 1257 張，SVM 的測試準確率從 59.2% 到 98.4%，kNN 從 59.6% 升到 98.4%。注意虛線是訓練準確率：樣本很少時它接近 100%，與測試之間拉開一道大縫——那就是 14.6 節要談的過度擬合，而「加資料」正是最直接的解藥，兩條線隨樣本增加而靠攏。右圖是更實用的視角：每加一批資料換到幾個百分點。從 25 加到 62 張換到 15.7 個百分點，但從 880 加到 1257 張只換到 0.44 個百分點。這個報酬遞減有兩個實務意涵：一是專案初期砸資源標資料非常划算，二是到了曲線平坦處，再標十倍資料也不會有驚喜，該去改善的是特徵、模型，或是資料的「品質」而不是「數量」——例如補齊那些模型一直分錯的困難樣本。

圖 14-4　資料是燃料：訓練集要多大才夠

14.3.1 MNIST：入門的經典

承叫獸開講，MNIST 是七萬張 28×28 灰階手寫數字(六萬訓練、一萬測試)，分 0 到 9 共十類。它體積小、乾淨、易上手，是所有人學習機器學習的「Hello World」，也是您課程中跨框架實作的主角。它的優點(簡單)也是它的侷限：現代模型在 MNIST 上輕鬆達到 99% 以上準確率，已無法區分演算法的優劣，因此僅適合入門教學，不再用於嚴肅的演算法評比。

14.3.2 CIFAR-10 與 CIFAR-100：邁向真實

CIFAR 資料集由多倫多大學整理，是六萬張 32×32 的彩色自然影像。CIFAR-10 分十類(飛機、汽車、鳥、貓、狗、船等),CIFAR-100 則細分為一百類。相較 MNIST 的黑白數字，CIFAR 是彩色的真實世界物體，有複雜的背景與多變的視角，難度大幅提升，是驗證中小型模型的常用基準。從 MNIST 到 CIFAR，反映了資料集「由簡入繁、

逼近真實」的演進軌跡。

14.3.3 ImageNet 與 ILSVRC：改變歷史的資料集

ImageNet 是一個劃時代的資料集，由史丹佛大學的李飛飛(Fei-Fei Li)團隊自 2009 年起建構，包含超過一千四百萬張影像、涵蓋兩萬多個類別，以人工標註。基於它舉辦的年度競賽 ILSVRC(ImageNet 大規模視覺辨識挑戰賽)，使用約一千個類別的子集，成為衡量視覺演算法的最高殿堂。

ImageNet 的歷史地位無可取代：2012 年，一個名為 AlexNet 的深度卷積神經網路在 ILSVRC 上以壓倒性差距奪冠，錯誤率大幅領先第二名，一舉引爆了深度學習革命(這正是第 15 章叫獸開講的主角)。可以說，沒有 ImageNet 這個規模空前的資料集，就沒有深度學習的爆發——它完美印證了「資料是燃料」的道理：是李飛飛「與其改進演算法、不如先建立夠大的資料集」的洞見，為 AlexNet 的成功鋪好了跑道。這也呼應本章的核心訊息：資料，常常比演算法更重要。

14.3.4 COCO：走向複雜場景

COCO(Common Objects in Context)由微軟推出，它的特點是「情境中的物件」：一張影像中包含多個物件、彼此有空間關係，且提供了比「這張圖是什麼」更豐富的標註——每個物件的精確位置(邊界框)、像素級輪廓(分割遮罩)、甚至人體關鍵點。因此 COCO 不只用於分類，更是物件偵測、實例分割、姿態估計的標準基準，直接銜接第 16 章的 YOLO。從 ImageNet(單物件分類)到 COCO(多物件偵測與分割)，又是一次「由簡入繁」的演進。

資料集	規模與尺寸	主要任務	歷史角色
MNIST	7 萬張，28×28 灰階	手寫數字分類	機器學習的 Hello World
CIFAR-10/100	6 萬張，32×32	自然物體分類	中小型模型基準

	彩色		
ImageNet	逾 1400 萬張	大規模分類(ILSVRC)	2012 引爆深度學習革命
COCO	逾 30 萬張	偵測、分割、姿態	複雜場景基準，接第 16 章

表 14-1 重要影像資料集比較

14.4 兩個經典分類器：kNN 與 SVM

14.4.1 k 最近鄰(kNN)

kNN 是最直觀的分類器，它的哲學是「物以類聚」：要判斷一個新樣本屬於哪一類，就看它在特徵空間中最靠近的 k 個鄰居，由這些鄰居「投票」決定——多數鄰居屬於哪一類，新樣本就歸為哪一類。例如 k=5，新樣本最近的五個鄰居中有三個是貓、兩個是狗，就判為貓。

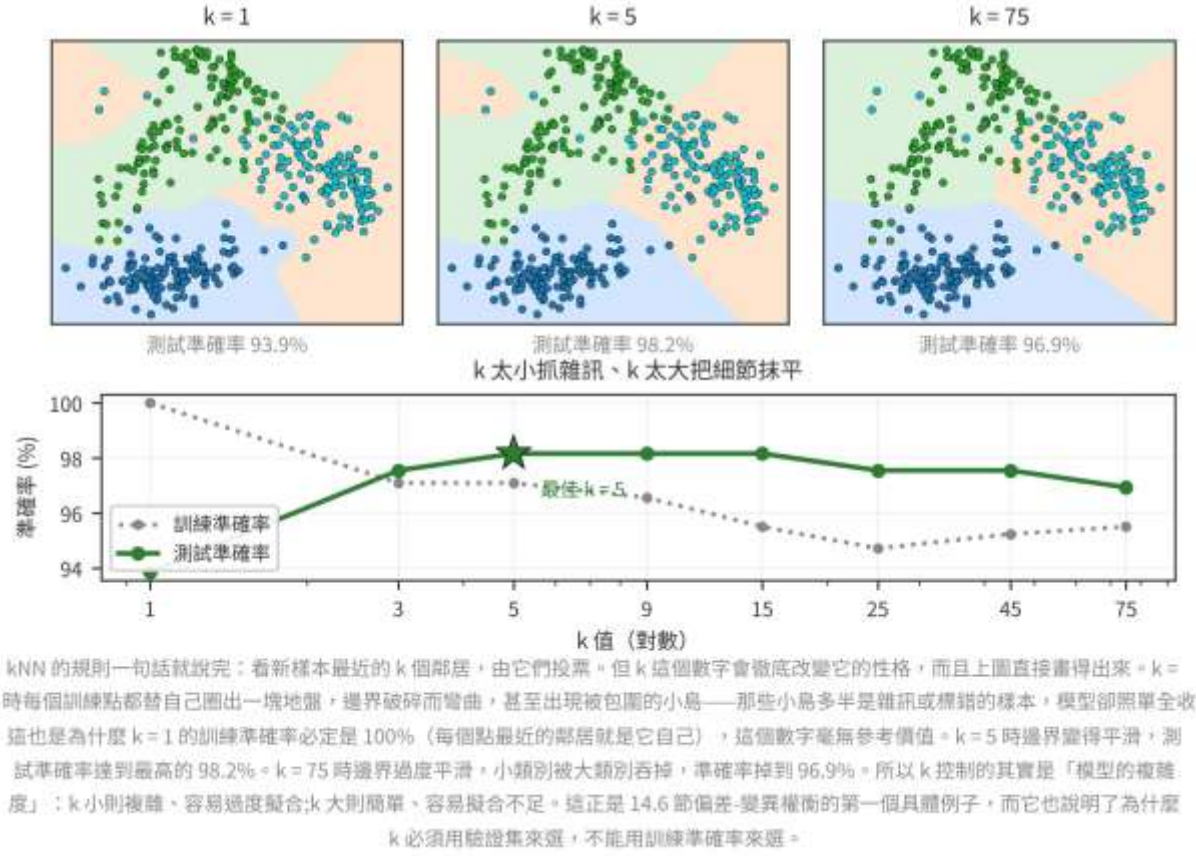
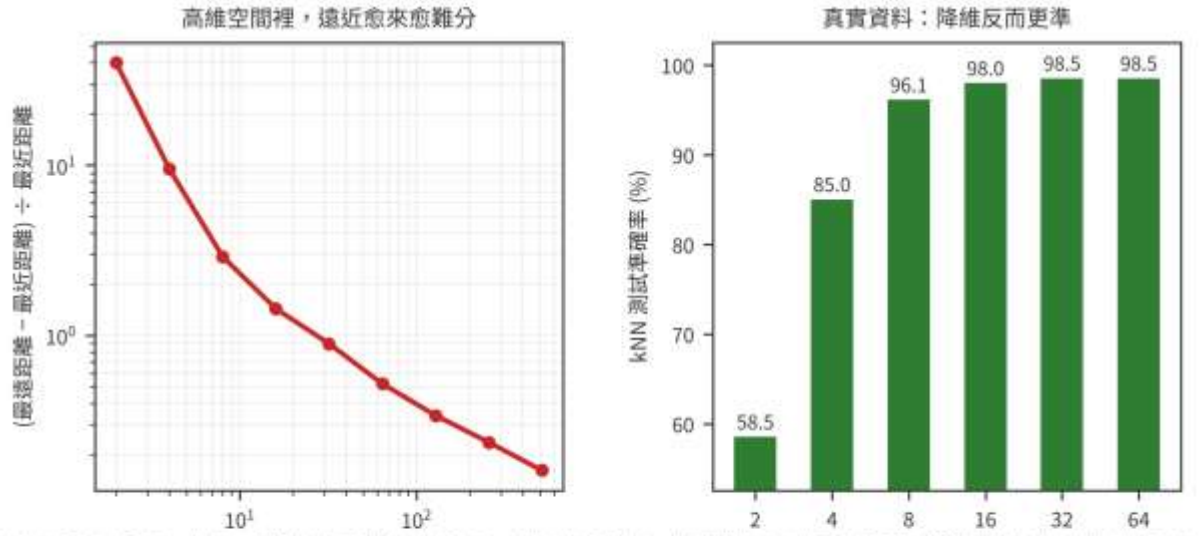


圖 14-5　kNN 的決策邊界與 k 值

kNN 有一個奇特的性質：它「不訓練」——沒有需要學習調整的參數，只是把所

有訓練樣本記住，預測時才逐一計算距離找鄰居。這使它實作極簡，但也帶來缺點：預測慢(每次都要與所有訓練樣本比距離)、記憶體耗費大(要存全部訓練資料)、且對特徵的尺度與維度敏感(高維空間中「距離」會失去意義，稱為維度災難)。k 值的選擇是關鍵：k 太小容易受雜訊影響，k 太大則會把界線模糊掉——這又是一個貫穿全書的取捨。kNN 常作為理解分類概念的第一站，以及新問題的簡單基準線。

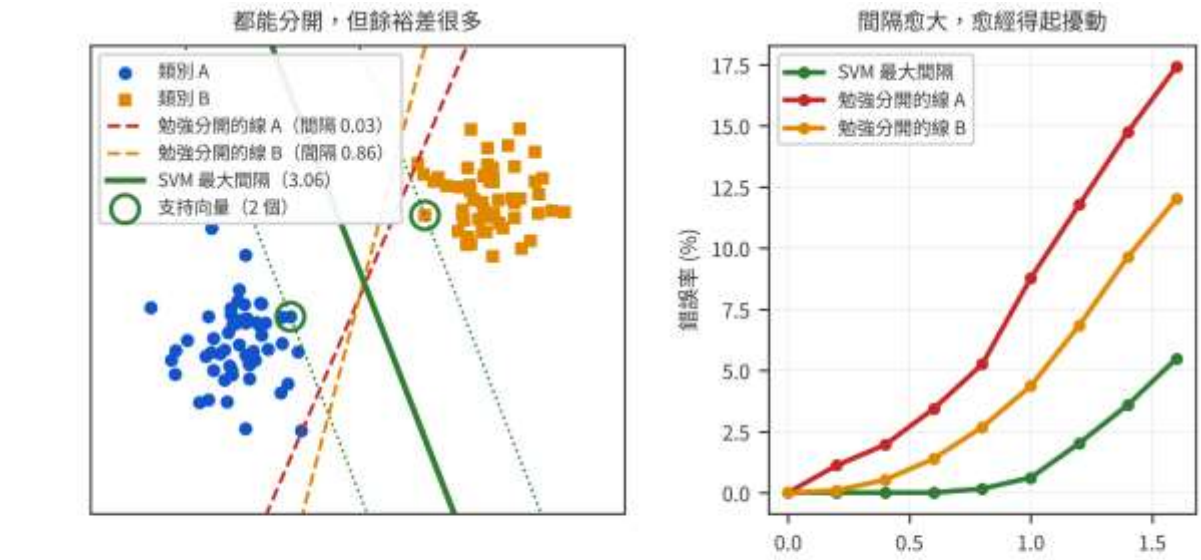


kNN 完全靠「距離」做事，所維度 D（對數）本身失去意義，它就失去了依據。左圖是純粹的數學現象：在 D 維單位立方體裡隨機撒 500 個點，再問「最遠的點比最近的點遠多少倍」。二維時這個比值是 40.0，但到了 512 維只剩 0.16——也就是說，所有點到查詢點的距離幾乎一樣遠，「最近的鄰居」與「最遠的鄰居」已經沒什麼差別了。這就是維度災難。右圖是同一件事在真實資料上的後果：把 64 維的手寫數字用 PCA 降到不同維度再跑 kNN，最佳落在 32 維（98.5%），比用完整 64 維的 98.5% 還高。這個結果常讓初學者意外：丟掉資訊反而更準。原因是被丟掉的多半是雜訊方向，保留的則是真正能區分類別的方向，距離因此變得更有意義。這也說明了第 13 章特徵工程的價值——好的特徵不只是「更有代表性」，也是「更低維」。

圖 14-6　維度災難：為什麼「最近」會失去意義

14.4.2 支持向量機(SVM)

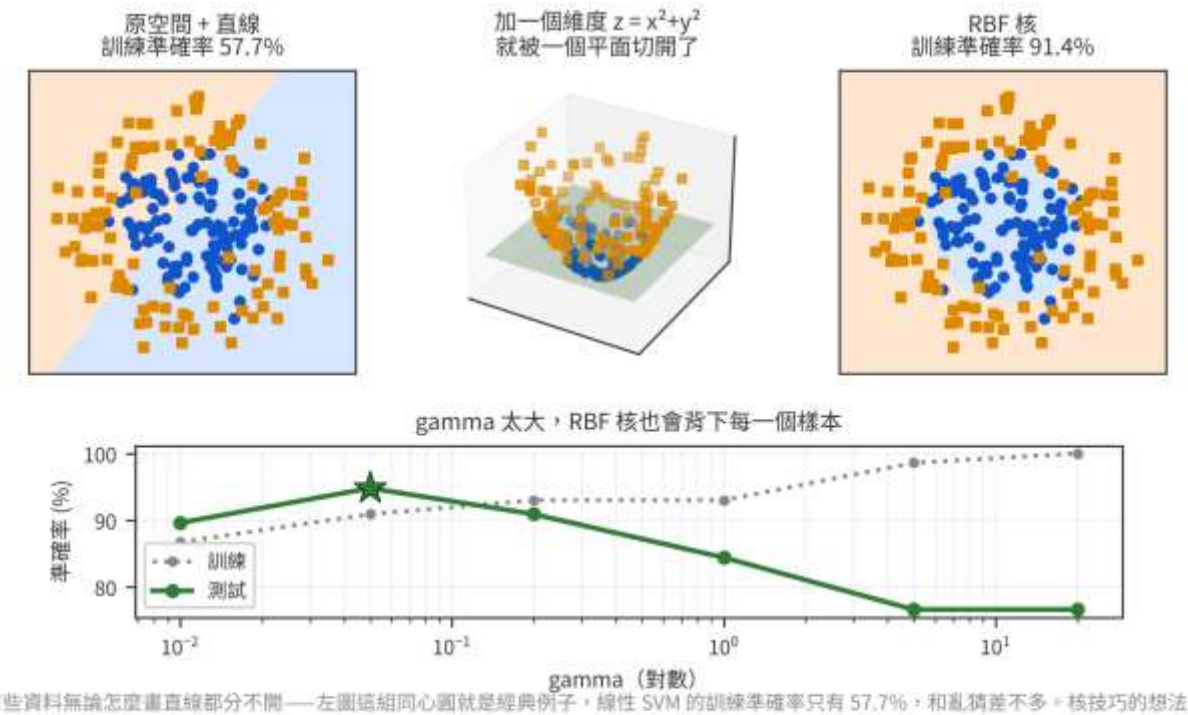
SVM 是深度學習興起前最強大、最受推崇的分類器之一。它的核心思想是找出一條「最好的分界線」(在高維空間中稱為超平面)來分開兩類資料。什麼叫「最好」?SVM 的答案很優雅：不只要能分開，還要讓分界線離兩邊最近的樣本盡可能遠——這個「最大間隔」(maximum margin)原則，讓分類器對新樣本更有容錯能力、泛化更好。那些恰好落在間隔邊界上、決定了分界線位置的關鍵樣本，就稱為「支持向量」，SVM 之名由此而來。



「最大間隔」為什麼有助於泛化？這張圖用實測回答。左圖的兩類資料有無限多條線可以分開，本圖測試資料的擾動強度0.4都完全正確地分開了訓練資料，但它們緊貼著某一邊的樣本，間隔只有 0.03、0.86；SVM 選的那條綠線間隔是 3.06，兩側各留出一條走廊。若只看訓練準確率，三條線一模一樣好，完全分不出高下。差別要在右圖才顯現：把測試樣本加上愈來愈大的擾動（模擬真實世界的拍攝差異、雜訊、個體變化），間隔小的線很快就開始出錯，而最大間隔那條撐得最久。道理很直觀——間隔就是「容錯的空間」，樣本在這條走廊的寬度之內怎麼晃動都還是分對的。順帶一提，決定這條線的只有貼在走廊邊上的那幾個樣本（本例 2 個，即支持向量），其餘樣本就算全部刪掉，答案也不會變——這是 SVM 模型可以很小的原因，也是 14.4.3 節要推導的重點。

圖 14-7　最大間隔：不只分得開，還要分得有餘裕

SVM 還有一個威力強大的祕密武器——核技巧(Kernel Trick)：當資料在原始空間無法用直線分開時(例如一類包在另一類中間)，核技巧能把資料「隱含地」映射到更高維的空間，在那裡它們就變得線性可分了，而且計算上不必真的算出高維座標。這讓 SVM 能處理複雜的非線性分類。在深度學習普及前，「人工設計特徵(如 SIFT/HOG)+ SVM 分類」是影像辨識的黃金組合，至今在樣本較少的場合仍是可靠選擇。



有些資料無論怎麼畫直線都分不開——左圖這組同心圓就是經典例子，線性 SVM 的訓練準確率只有 57.7%，和亂猜差不多。核技巧的想法是：既然在二維分不開，就升到三維看看。中圖把每個點加上第三個座標 $z = x^2 + y^2$，內圈的點 z 小、外圈的點 z 大，於是一個水平的平面就乾淨地把兩類切開了。回到二維看，這個平面就是一個圓。「核技巧」的巧妙在於：它根本不需要真的把資料升上去。SVM 的推導中資料只以內積的形式出現，所以只要能直接算出「升維之後兩點的內積」就夠了，而核函數正是在做這件事——省掉了升維的運算與儲存，甚至可以升到無限維（RBF 核就是）。右圖的 RBF 核把訓練準確率拉到 91.4%。但天下沒有白吃的午餐：下圖掃描 gamma 顯示，gamma 太大時訓練準確率衝到 100%、測試卻掉到 76.6%——核把模型的能力放大了，也把過度擬合的風險一起放大了。

圖 14-8　核技巧：在原空間分不開，升上去就分得開

程式 14-1 kNN 與 SVM 手寫數字分類

```
import cv2, numpy as np
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, confusion_matrix

# 載入手寫數字(類 MNIST 的小型資料集)
digits = load_digits()
X = digits.data          # 每張影像攤平成特徵向量
y = digits.target        # 標籤 0~9

# 切分訓練/測試(14.5 節的核心觀念)
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.2, random_state=42)

# --- kNN ---
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_tr, y_tr)              # kNN 的 fit 只是記住資料
pred_knn = knn.predict(X_te)
print("kNN 準確率:", accuracy_score(y_te, pred_knn))

# --- SVM(RBF 核) ---
```

```
svm = SVC(kernel="rbf", C=10, gamma=0.001)
svm.fit(X_tr, y_tr)
pred_svm = svm.predict(X_te)
print("SVM 準確率:", accuracy_score(y_te, pred_svm))
print("SVM 混淆矩陣:\n", confusion_matrix(y_te, pred_svm))
```

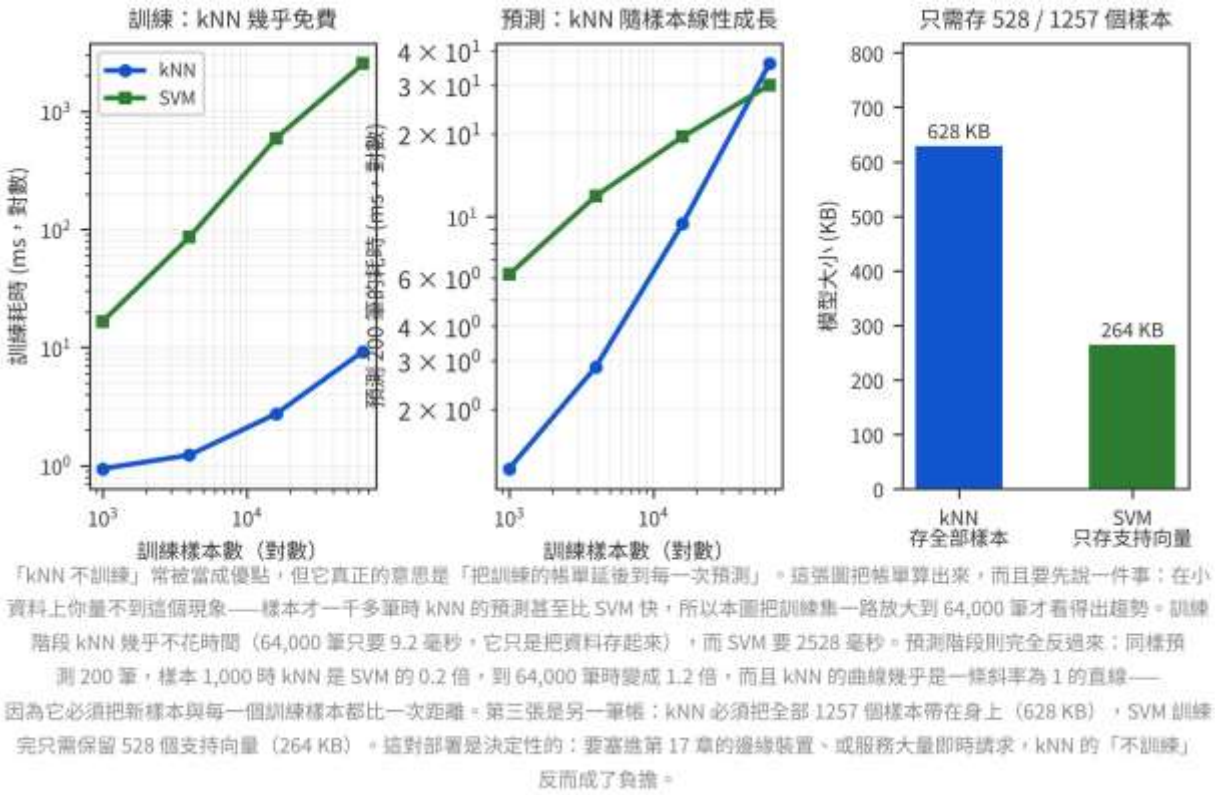


圖 14-9　kNN 的帳單：不訓練，但每次預測都要付錢

14.4.3 推導：最大間隔為什麼等價於最小化 |w|

14.4.2 節說 SVM 要找「離兩邊最近樣本盡可能遠」的分界線，這句話聽起來像個模糊的審美標準。但它其實可以寫成一道有唯一解的最佳化問題，而且推導只需要高中的點到直線距離公式。值得花十分鐘跟著走一遍，因為這是全書第一次完整示範「把一個直覺要求翻譯成數學目標」——第 15 章的損失函數、第 16 章的 IoU 門檻，用的都是同一套思路。

先把分界線寫成一般式。在二維平面上它是一條直線 w1·x1 + w2·x2 + b = 0;在 d 維特徵空間中則是一個超平面，用向量內積寫成 w·x + b = 0，其中 w 是這個超平面的法向量(垂直於它的方向)、b 決定它離原點多遠。任一點 x 到這個超平面的距離，正是高中那條公式的推廣：距離 = |w·x + b| / |w|，其中 |w| = √(w1² + w2² + … + wd²) 是

法向量的長度。

接下來是整個推導最關鍵、也最常被跳過的一步：縮放的自由度。請注意 w·x + b = 0 與 (3w)·x + 3b = 0 畫出來是同一個超平面——把 w 和 b 同時乘上任何非零常數，幾何上什麼都沒變。這表示 (w, b) 這組參數有一個多餘的自由度，我們可以挑一個最方便的縮放，而不損失任何一般性。SVM 選的是：讓離超平面最近的那些樣本恰好滿足 |w·x + b| = 1。這個約定叫做「正規化超平面」，它不是額外的假設，只是把尺規訂死而已。

尺規訂死之後，一切就都量得出來了。設每個樣本的標籤 y 只取 −1 或 +1(SVM 習慣用 ±1 而非 0/1，等一下就知道為什麼)，則「全部樣本都被正確分類、且最近的樣本剛好落在 ±1 上」可以濃縮成一條不等式：對每一筆樣本 (x, y) 都有 y·(w·x + b) ≥ 1。這就是用 ±1 當標籤的好處：正樣本要求 w·x + b ≥ 1、負樣本要求 w·x + b ≤ −1，各自乘上自己的 y 之後，兩者合而為一。

而最近樣本到超平面的距離，依前面的公式就是 |w·x + b| / |w| = 1 / |w|。兩側各有一條這樣的邊界，所以「間隔」(margin，兩條邊界之間的總寬度)等於 2 / |w|。到這裡，那個模糊的審美標準就變成了一個精確的目標：最大化 2 / |w|。

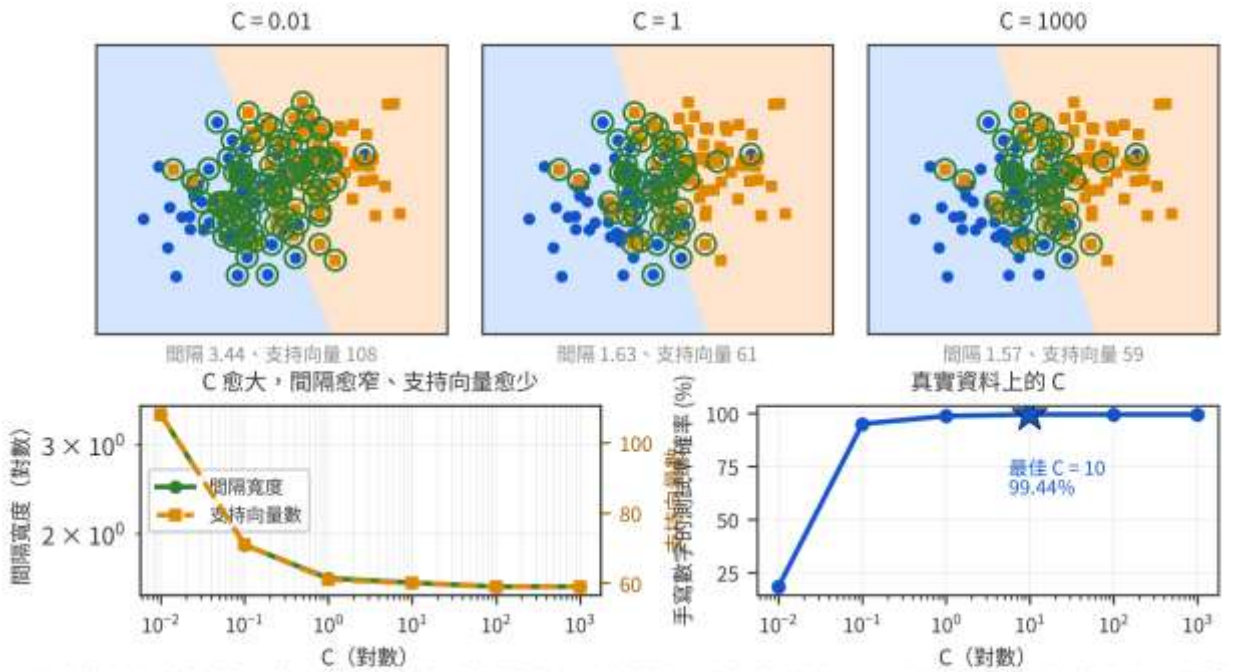
最後是兩次等價轉換。其一，最大化 2 / |w| 等同於最小化 |w|，因為分子是常數，而 |w| > 0 時兩者單調反向。其二，最小化 |w| 等同於最小化 ½·|w|²，因為平方在正數上單調遞增，不改變最小值的位置;加上 ½ 純粹是為了微分之後係數剛好消掉，是數學家的小小潔癖。於是整個問題寫成：

```
最小化    (1/2) · |w|²
受限於    y · (w·x + b) ≥ 1 ,   對每一筆訓練樣本 (x, y)
```

這個形式有三個漂亮的性質，也是 SVM 在 1990 年代席捲學界的原因。第一，目標函數是凸函數、約束是線性的，所以這是一個凸二次規劃問題——它只有一個全域最佳解，不像神經網路那樣會卡在局部極小值;同一批資料跑一百次會得到同一個答案。

第二，用拉格朗日乘子法解它的對偶問題時，會發現最終的 w 只由那些「不等式取等號」的樣本決定，其餘樣本的乘子都是 0——這就是支持向量的數學來源，也解釋了圖 14-7 的實測：110 個樣本中只有 2 個是支持向量。第三，對偶形式中樣本只以內積的形式出現，把內積換成核函數就得到了非線性的 SVM——這正是 14.4.2 節核技巧「不必真的算出高維座標」的根據，也是圖 14-8 能把線性核的 57.7% 拉到 RBF 核 91.4% 的原理。

最後補一個實務上的轉折。上面的推導假設資料完全線性可分，但真實資料幾乎從不如此，只要有一個離群樣本就會讓約束無解。因此實用的 SVM 在每個約束上放寬一個鬆弛量 $\xi \ge 0$，改成 $y\cdot(w\cdot x + b) \ge 1 - \xi$，並在目標函數裡加上懲罰項，變成最小化 $(1/2)\cdot|w|^2 + C\cdot\Sigma\xi$。這個 C 就是 14.4.2 節與圖 14-10 裡那個參數：C 很大時懲罰很重，模型不准犯錯、間隔被壓窄;C 很小時容許犯錯、間隔變寬。圖 14-10 的實測正是這條公式的具體樣貌：C 一路加大，間隔從 3.44 縮到 1.57、支持向量從 108 個減到 59 個。所以 C 不是一個神祕的旋鈕，它就是「間隔」與「訓練誤差」之間的兌換率——而這個兌換率該訂在哪裡，只能靠驗證集決定，不能靠訓練集。



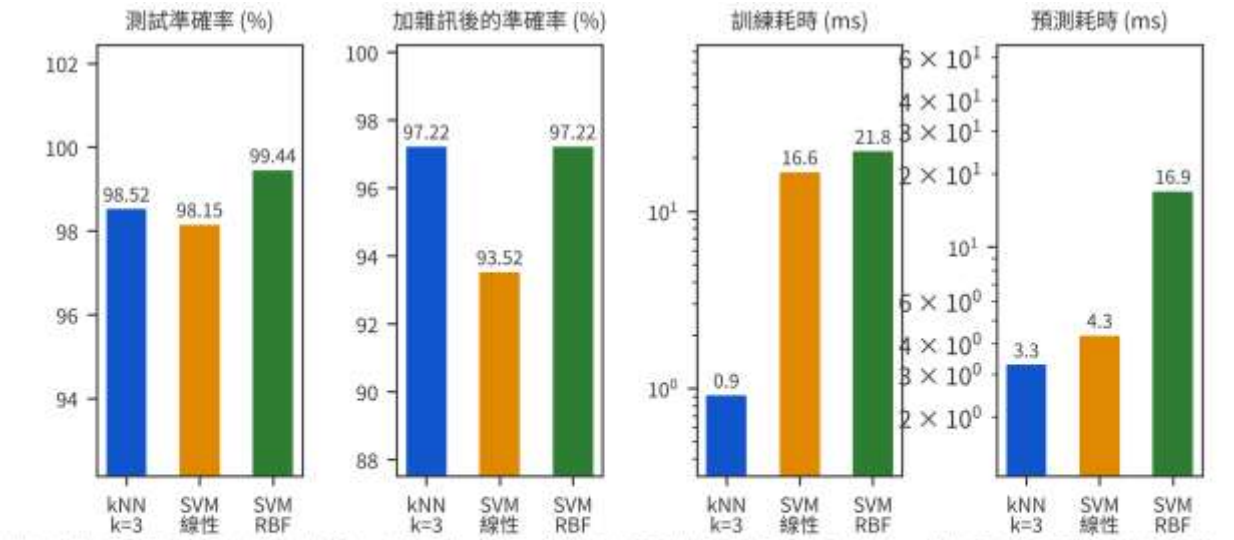
現實的資料不會乖乖分開，總會有幾個標錯或天生就長得像另一類的樣本。C 這個參數決定了 SVM 要用多強硬的態度面對它們：C 是「犯錯的罰金」。罰金低（C = 0.01）時，模型寧願放過幾個難搞的樣本，換取一條又寬又平滑的走廊，此時間隔 3.44、支持向量多達 108 個;罰金高（C = 1000）時，模型會為了遷就每一個樣本而把邊界扭過去，間隔縮到 1.57、支持向量只剩 59 個。左下的兩條曲線把這個趨勢量了出來。右下是真實資料上的後果：在手寫數字上掃 C，最佳落在 C = 10（99.44%），兩端都比較差。C 太小是擬合不足（連該學的都沒學），C 太大是過度擬合（連雜訊都背了下來）——這又是一次偏差與變異的拉鋸，而拉鋸的位置必須用驗證集決定，不能憑經驗填一個數字。

圖 14-10　C 參數：間隔要寬，還是不准錯

14.5 資料切分與模型評估

14.5.1 訓練、驗證、測試：為什麼要三分

這是機器學習最重要、也最容易被初學者忽略的觀念：評估模型，絕不能用它訓練時看過的資料。道理很簡單——一個學生若考的是他背過答案的題目，高分不代表他真懂。模型也一樣：它可能只是「記住」了訓練資料，而非「學會」了規律。因此我們把資料切成三份：



把兩個經典分類器放在同一批手寫數字上，用四個軸一起看。準確率上 RBF 核的 SVM 最高（99.44%），線性 SVM 98.15%，kNN 98.52%，三者其實都很接近——在這種乾淨、置中、類別清楚的資料上，選哪個都不會差太多。差別要在其他軸上才看得出來。第二格是把測試影像加上雜訊之後的表現：kNN 掉到 97.22%、RBF SVM 掉到 97.22%，因為 kNN 完全靠距離，任何擾動都直接反映在距離上。第三、四格是成本：kNN 的訓練幾乎免費（0.9 毫秒），SVM 要 21.8 毫秒。實務上的選擇原則因此很清楚：樣本少、類別可能隨時增加、需要「它像哪幾個樣本」這種解釋，選 kNN;樣本中等、要穩定的邊界、要小模型可以部署，選 SVM;而一旦樣本多到數十萬、特徵是原始像素，兩者都會被第 15 章的神經網路拉開——不是因為它們不好，而是因為它們的特徵得靠人去想。

圖 14-11　擂台：kNN 與 SVM 在五個軸上的對照

1. 訓練集(Training Set)：用來調整模型參數，佔最大比例(如 60~80%)。
2. 驗證集(Validation Set)：在訓練過程中用來調整「超參數」(如 kNN 的 k、SVM 的 C)、選擇模型、決定何時停止訓練。它不直接參與參數調整，但會影響我們的決策。
3. 測試集(Test Set)：模型與所有決策都定案後，最後才用它評估——它模擬「模型上線後遇到全新資料」的真實表現。測試集必須嚴格保密，絕不能在開發過程中反覆使用，否則就等於間接讓模型「看過」了它，評估結果會失真、過度樂觀。

這也解釋了叫獸開講中 MNIST 的設計巧思：LeCun 團隊刻意混合「人口普查局員工(易)」與「高中生(難)」兩種來源，並確保訓練集與測試集的難度分布一致——若訓練集全是工整字跡、測試集全是潦草字跡，模型的測試表現會低估它的真實能力，反之則高估。資料切分的公平性，是可靠評估的前提。當資料量較少時，還可用「交叉驗證」(把資料輪流當訓練與驗證，取平均)來更充分地利用資料。

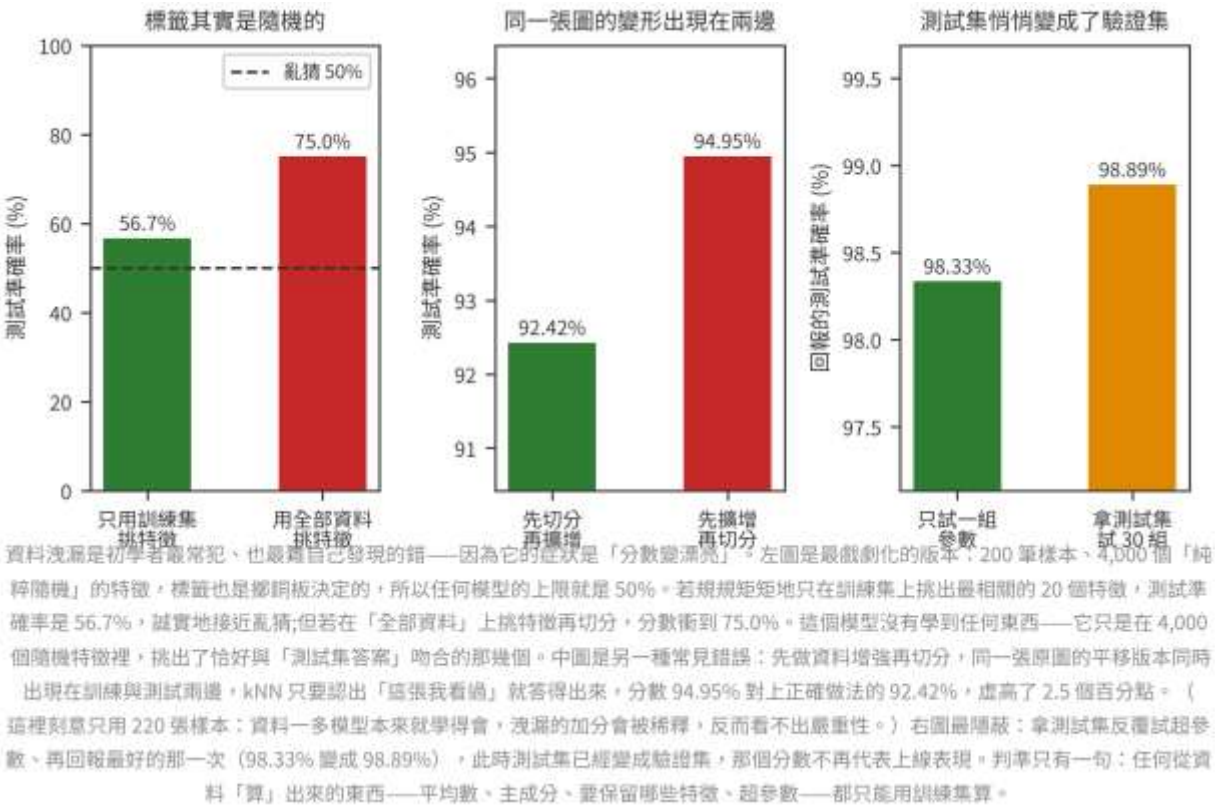
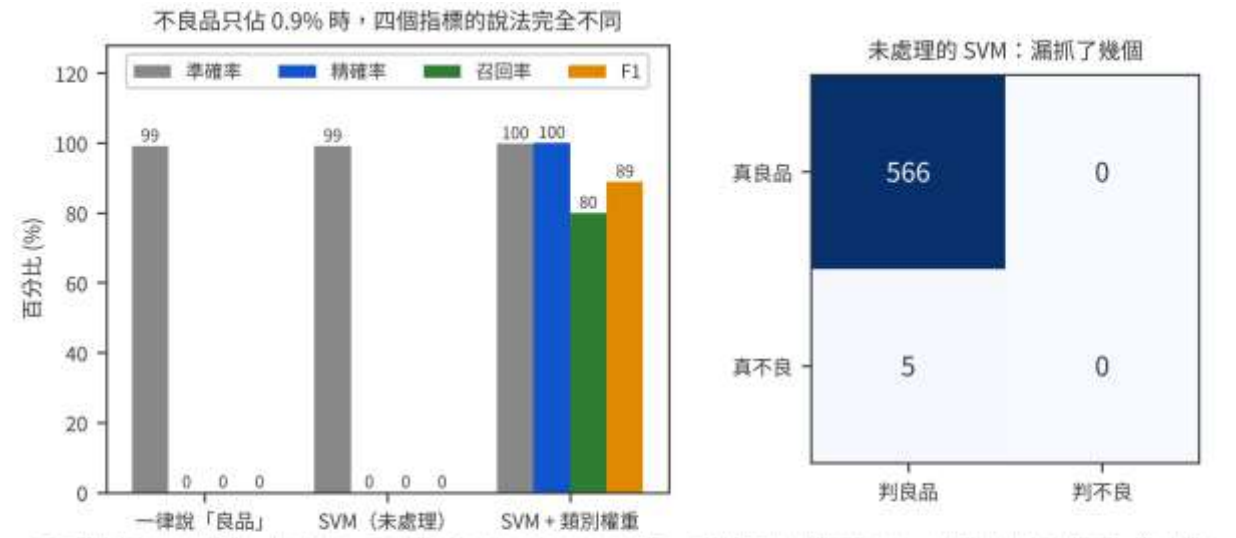


圖 14-12　資料洩漏：測試集被偷看會發生什麼事

14.5.2 評估指標：準確率的陷阱

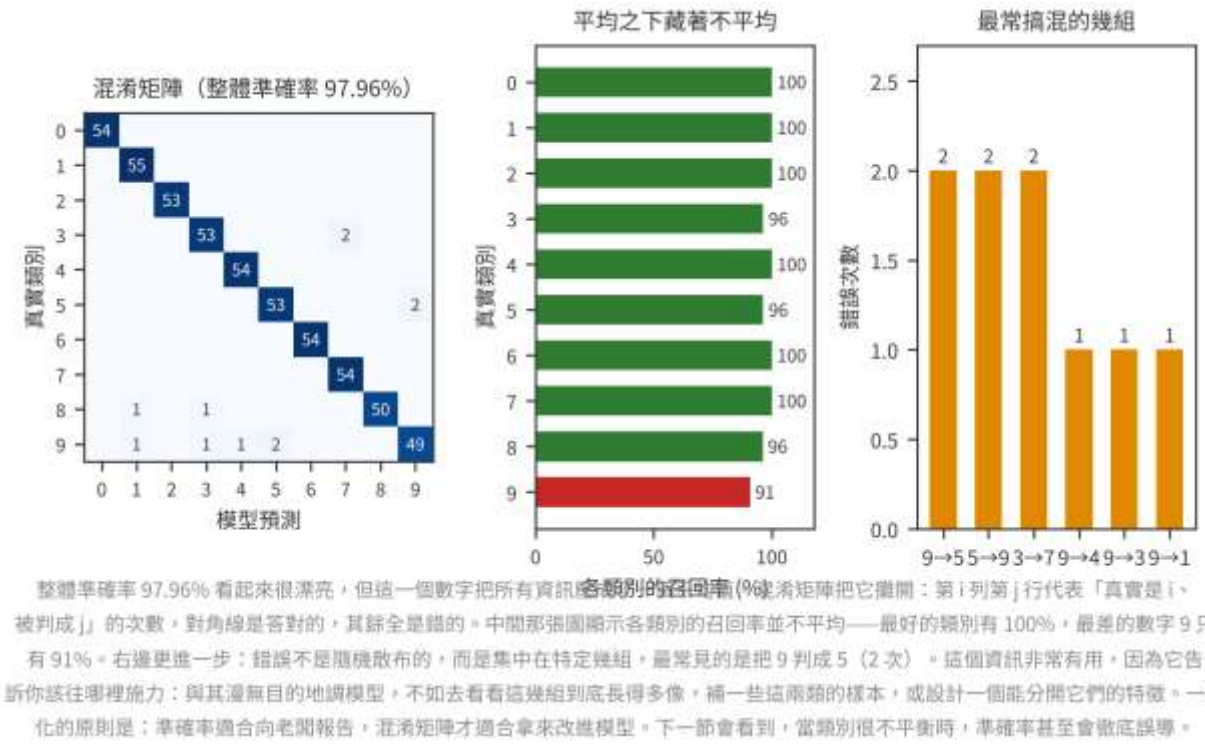
最直觀的指標是準確率(Accuracy)：預測正確的比例。但準確率有一個著名的陷阱——類別不平衡時會嚴重誤導。舉例：某罕見疾病的盛行率是 1%，一個「無論如何都判斷沒病」的模型，準確率高達 99%，卻毫無用處(它抓不出任何一個病人)。因此，只看準確率是危險的，必須輔以更細緻的指標。這裡要補一個比「一律判沒病」更令人不安的事實：不是只有偷懶的基準模型才會這樣，一個認真訓練的分類器在類別極度不平衡時也會做出同樣的選擇，因為「最小化整體錯誤」最快的方法就是放棄少數類。圖 14-13 的實測：不良品只佔 0.86% 時，「一律說良品」與一個未做任何處理的 SVM，兩者的準確率都是 99.12%、召回率都是 0%——也就是說，那個 SVM 花了時間訓練，學到的東西與什麼都不學完全一樣。加上類別權重之後，準確率 99.82%、召回率才升到 80%。



這張圖把「準確率的陷阱」做成實驗。我們把數字 8 當成產線上的不良品，刻意把它的比例壓到 0.9%，模擬真實的品檢場景。第一根柱子是一個什麼都不做的「模型」：一律回答良品。它的準確率高達 99.1%，漂亮得可以寫進簡報——但它的召回率是 0%，一個不良品都沒抓到，完全沒有用。這就是類別不平衡時準確率會誤導的原因：多數類別把平均值整個拉上去了。更值得警惕的是中間那根：一個正常訓練的 SVM，準確率 99.1% 甚至比「一律說良品」還高，但它的召回率同樣是 0%——模型在訓練時發現「全部猜多數類別」就能把損失壓得很低，於是它就真的這麼做了。這不是模型壞掉，是我們給錯了目標。第三根只改一件事：加上類別權重，讓罕見類別的錯誤罰得更重（權重與類別頻率成反比）。召回率立刻升到 80%、精確率 100%，而準確率 99.8% 幾乎沒變——這正說明了準確率對這件事有多麼不敏感。要選哪一個，取決於代價：漏抓一個不良品流到客戶手上，與誤判一個良品重驗一次，這兩件事的損失通常差好幾個數量級。指標不是數學問題，是商業問題。

圖 14-13　準確率的陷阱：類別不平衡

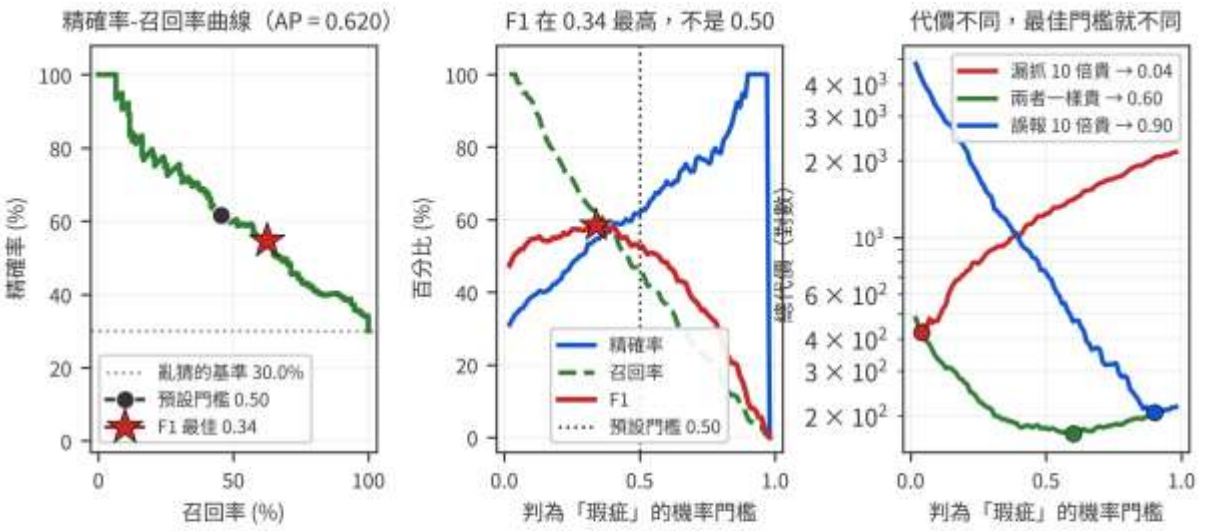
混淆矩陣(Confusion Matrix)是評估的基礎工具：它是一張表格，列出「實際類別」與「預測類別」的所有組合，清楚顯示模型把哪些類別混淆了(例如把 4 誤判成 9)。從混淆矩陣可導出兩個關鍵指標：精確率(Precision，模型說是陽性的裡面，有多少真的是陽性——衡量「不誤報」)與召回率(Recall，所有真正的陽性中，模型抓出了多少——衡量「不漏抓」)。這兩者常有取捨：提高召回(寧可錯抓也不放過)往往降低精確(誤報變多)，反之亦然;F1 分數則是兩者的調和平均，綜合衡量。



整體準確率 97.96% 看起來很漂亮，但這一個數字把所有資訊壓成一個。混淆矩陣把它攤開：第 i 列第 j 行代表「真實是 i、被判成 j」的次數，對角線是答對的，其餘全是錯的。中間那張圖顯示各類別的召回率並不平均——最好的類別有 100%，最差的數字 9 只有 91%。右邊更進一步：錯誤不是隨機散布的，而是集中在特定幾組，最常見的是把 9 判成 5（2 次）。這個資訊非常有用，因為它告訴你該往哪裡施力：與其漫無目的地調模型，不如去看看這幾組到底長得多像，補一些這兩類的樣本，或設計一個能分開它們的特徵。一般化的原則是：準確率適合向老闆報告，混淆矩陣才適合拿來改進模型。下一節會看到，當類別很不平衡時，準確率甚至會徹底誤導。

圖 14-14　混淆矩陣：一個數字看不出來的事

選擇哪個指標，取決於應用的代價結構——這是工程判斷，不是數學問題。癌症篩檢重召回(漏抓一個病人的代價，遠大於多做幾次複檢);垃圾郵件過濾重精確(把重要郵件誤判為垃圾的代價，大於偶爾漏掉一封垃圾);這與第 10 章 Viola-Jones 的「漏檢 vs. 誤報」、第 9 章 Canny 雙門檻，是同一類「依代價權衡」的思維，貫穿整本書。還要補上一件實務上最常轉、教科書卻最少提的旋鈕：判定門檻。分類器真正輸出的是一個分數或機率，「判為陽性」是我們拿 0.5 去切出來的，而那個 0.5 與任務的代價毫無關係。門檻調低，召回率上升、精確率下降;調高則反之。圖 14-15 的實測顯示，同一個模型、同一批資料，在三種代價結構下的最佳門檻分別是 0.04、0.60 與 0.90;即使只看 F1，最佳門檻也是 0.34 而非 0.50，差了 6.0 個百分點。換句話說，調門檻不是技術細節，而是把商業判斷寫進系統的那一步，而且這一步只有懂現場的人能做。



分類器真正輸出的是一個分數或機率，「判為瑕疵」是我們拿 0.5 去切出來的——而那個 0.5 從來不是什麼神聖的數字。中圖把門檻從 0.02 掃到 0.98：門檻低時幾乎全判瑕疵，召回率接近 100% 而精確率很低（寧可錯殺）；門檻高時反過來（寧可放過）。兩條線必然交叉，F1 在 0.34 處達到 58.3%，而預設的 0.50 只有 52.3%（精確率 61.6%、召回率 45.4%）——光是轉一下這個旋鈕就多了 6.0 個百分點，一行模型程式都不用改。左圖是同一件事的另一種畫法：精確率-召回率曲線上的每一點就是一個門檻，曲線下的面積（AP = 0.620）因此是一個與門檻無關的總體指標，比任何單一門檻下的準確率都可靠，也是第 16 章物件偵測評分的基礎。右圖才是最該帶走的一課：最佳門檻取決於代價。漏抓的代價是誤報十倍時（瑕疵品流到客戶手上），最佳門檻低到 0.04；兩者一樣貴時是 0.60；誤報十倍貴時（誤判害整批重工）升到 0.90。同一個模型、同一批資料，答案卻完全不同。所以調門檻不是技術細節，而是把商業判斷寫進系統的那一步，而且這一步只有懂現場的人能做，不是調參程式能自動決定的。

圖 14-15　門檻不是 0.5：精確率、召回率與代價

14.5.3 推導：一個準確率該附上多大的誤差範圍

14.5.1 節說資料量少時可以用交叉驗證，圖 14-16 則實測了單次切分有多不穩。這一小節補上背後的算式，讓你能在任何時候自己估出「我報的這個數字，誤差大概多少」——這是一個只需要兩個數字就算得完、卻幾乎沒有人去算的量。

先把問題說清楚。測試集有 n 筆樣本，模型答對了 k 筆，我們報告的準確率是 a = k / n。但我們真正想知道的不是 a，而是這個模型在「所有可能的新樣本」上的真實準確率 p——a 只是 p 的一次抽樣估計。如果把每一筆測試樣本看成一次獨立的伯努利試驗(答對的機率是 p、答錯是 1 − p)，那麼答對的總數 k 服從二項分布，其變異數為 n·p·(1 − p)。

把它除以 n^2 換算成 a 的變異數，再開根號，就得到準確率估計的標準誤：

```
SE(a) = √( p · (1 − p) / n )  ≈  √( a · (1 − a) / n )
約 95% 信賴區間 ：  a − 1.96·SE(a)  ～  a + 1.96·SE(a)
```

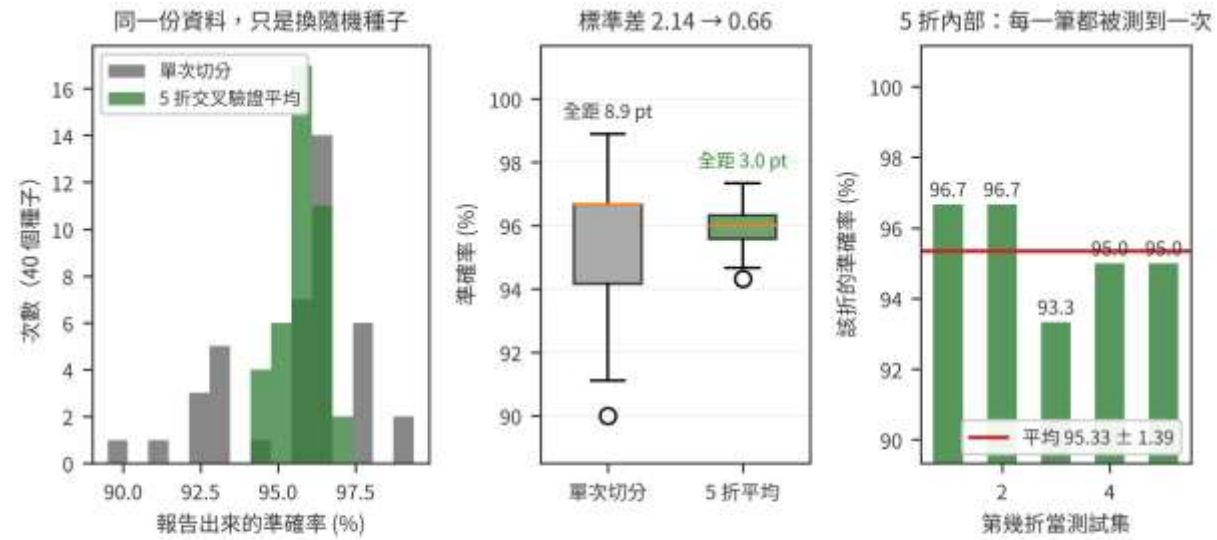
這條公式有三個推論值得記住。其一，標準誤隨 √n 下降：要把誤差砍半，測試

集必須變成四倍大。其二，$p\cdot(1-p)$ 在 $p = 0.5$ 時最大、在 p 接近 0 或 1 時最小，所以同樣大小的測試集，高準確率的模型估得比較準。其三，也是最實用的：拿 1.96 倍的標準誤往兩邊各畫一道，就是那個該寫進報告裡的區間。

代幾個數字就會冒冷汗。測試集 $n = 200$、準確率 $a = 0.90$ 時，$SE = \sqrt{(0.9 \times 0.1 / 200)} \approx 0.0212$，95% 信賴區間是 90% ± 4.2%，也就是 85.8% 到 94.2%。換句話說，在一個 200 筆的測試集上，一個 91% 的模型與一個 89% 的模型根本分不出高下。而論文與驗收報告裡「我們的方法比基準高 1.5 個百分點」這種句子，如果測試集只有兩三百筆，那 1.5 個百分點完全埋在雜訊裡。$n = 2{,}000$ 時 SE 降到 0.67%，區間縮成 88.7% 到 91.3%;$n = 20{,}000$ 時才來到 ±0.42%。這就是為什麼嚴肅的演算法評比，測試集動輒上萬筆。

交叉驗證處理的則是另一個誤差來源。上面的公式只算了「測試集抽樣」的隨機性，卻假設模型本身是固定的;但換一次切分，訓練集也跟著變，學出來的模型就不一樣了。圖 14-16 量到的就是這一層：同一份 300 筆資料、同一個模型設定，光是換 40 個隨機種子，單次切分的結果就從 90.0% 跳到 98.9%，標準差 2.14 個百分點。k 折交叉驗證把資料切成 k 份、輪流拿一份當測試集，每一筆樣本都剛好被測到一次，再把 k 次結果平均;實測顯示這樣得到的估計標準差降到 0.66，穩定了約 3.3 倍。

報告交叉驗證結果時，慣例是寫成「平均 ± 標準差」，例如 95.33% ± 1.39%。要注意這個標準差是「折與折之間」的變異，它同時反映了測試集抽樣與訓練集變動兩種來源，因此通常比上面那條二項分布公式算出來的標準誤大——而這是誠實的，因為它涵蓋了更多真實存在的不確定性。順帶一提，k 折的各折並不獨立(各折的訓練集彼此高度重疊)，所以嚴格說來不能直接把標準差除以 $\sqrt{k}$ 當成標準誤使用;這是統計上一個著名的細節，但在初學階段，只要養成「報平均也要報標準差」的習慣，就已經勝過大多數人了。



「我的模型測試準確率 96.7%」這句話，比多數人以為的更不可靠。做法是用同一份資料（300 筆）、同一個模型，只換 40 個隨機種子，各做兩件事：一次 7:3 切分，以及一次 5 折交叉驗證。灰色那堆是單次切分的結果：從 90.0% 到 98.9%，全距 8.9 個百分點，標準差 2.14。換句話說，只要種子換得好，同一個模型就能「變好」好幾個百分點，而這種「調種子」在論文與驗收報告裡比想像中常見。綠色那堆是 5 折平均：全距縮到 3.0 個百分點，標準差 0.66，也就是穩定了約 3.3 倍。比較模型時問題更嚴重：本圖實測 SVM 與 kNN 在這 40 次切分中，有 13 次 kNN 反而贏——只切一次就下結論，有相當機率選錯模型。右圖是 5 折內部的樣子：每一折輪流當測試集，折與折之間本來就差 3.3 個百分點，平均之後才是 95.33% ± 1.39。實務上的規矩因此很簡單：資料量不大時一律用交叉驗證，而且報告準確率時要附上標準差——沒有誤差範圍的單一數字，在統計上幾乎沒有意義。

圖 14-16　一次切分不可信：交叉驗證把變異壓下來

14.6 過度擬合、擬合不足與偏差-變異權衡

這是機器學習最核心的理論觀念，也是所有實務調校的指南針。它回答一個根本問題：為什麼模型在訓練資料上表現很好，到了真實世界卻不行?

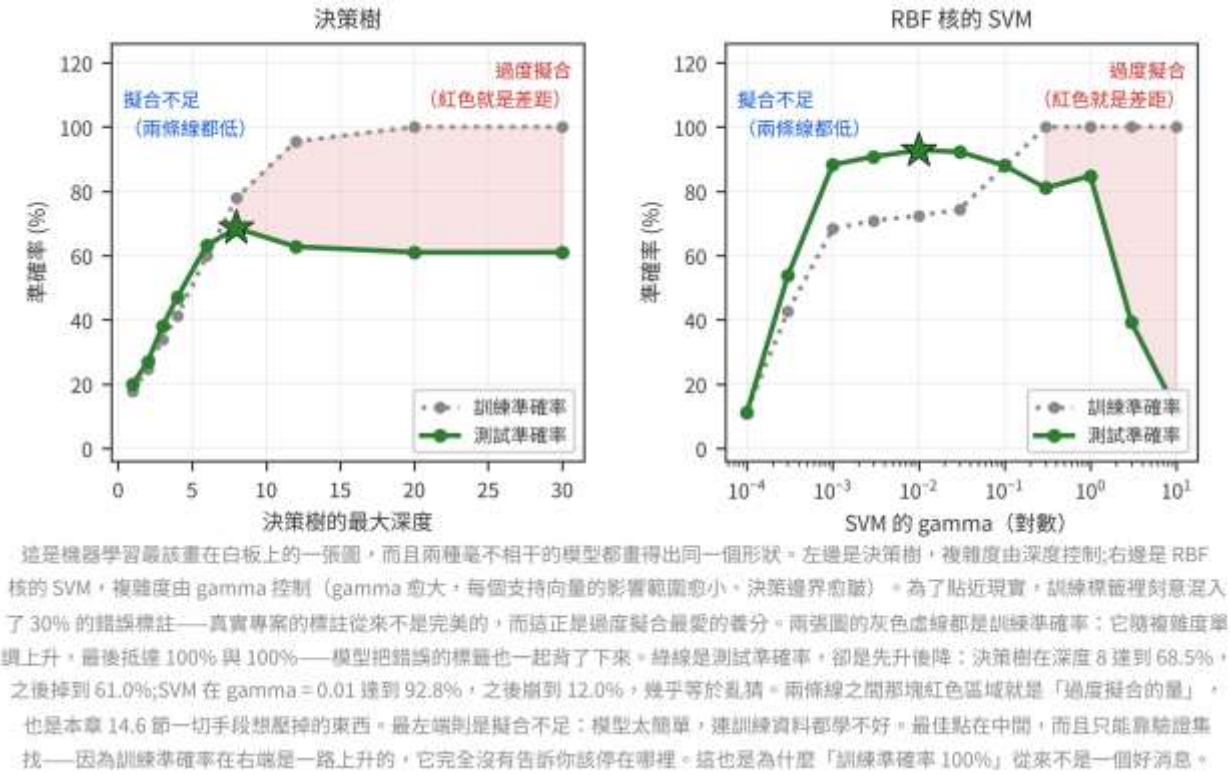
14.6.1 擬合不足與過度擬合

擬合不足(Underfitting)是模型太簡單，連訓練資料的規律都抓不住——訓練表現與測試表現都差。好比用一條直線去擬合明顯彎曲的資料，怎麼畫都不貼。

過度擬合(Overfitting)則相反，是模型太複雜，把訓練資料「背」得太死，連其中的雜訊與偶然細節都當成規律學了進去——訓練表現極好，但一遇到沒看過的測試資料就崩潰。好比一個學生死背了考古題的每個答案，換個題目就不會了。過度擬合是實務中最常見、最需要警惕的問題。

診斷方法很簡單：比較模型在訓練集與驗證集上的表現。兩者都差是擬合不足;訓練好、驗證差且落差很大，就是過度擬合。理想的模型落在中間——複雜度剛好抓住

真實規律，又不至於背下雜訊。

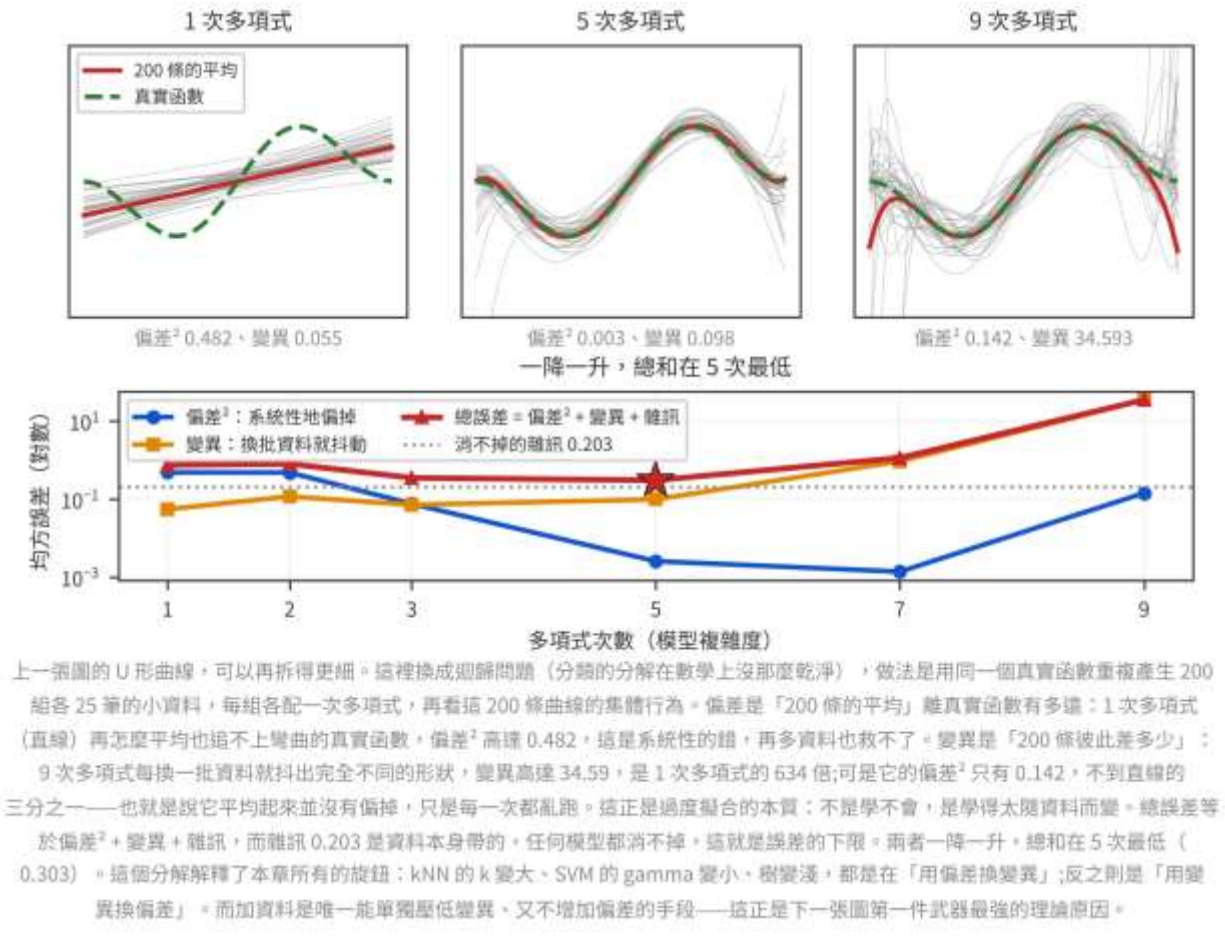


這是機器學習最該畫在白板上的一張圖，而且兩種毫不相干的模型都畫得出同一個形狀。左邊是決策樹，複雜度由深度控制;右邊是 RBF 核的 SVM，複雜度由 gamma 控制（gamma 愈大，每個支持向量的影響範圍愈小、決策邊界愈嚴）。為了貼近現實，訓練標籤裡刻意混入了 30% 的錯誤標註——真實專案的標註從來不是完美的，而這正是過度擬合最愛的養分。兩張圖的灰色虛線都是訓練準確率：它隨複雜度單調上升，最後抵達 100% 與 100%——模型把錯誤的標籤也一起背了下來。綠線是測試準確率，卻是先升後降：決策樹在深度 8 達到 68.5%，之後掉到 61.0%;SVM 在 gamma = 0.01 達到 92.8%，之後崩到 12.0%，幾乎等於亂猜。兩條線之間那塊紅色區域就是「過度擬合的量」，也是本章 14.6 節一切手段想壓掉的東西。最左端則是擬合不足：模型太簡單，連訓練資料都學不好。最佳點在中間，而且只能靠驗證集找——因為訓練準確率在右端是一路上升的，它完全沒有告訴你該停在哪裡。這也是為什麼「訓練準確率 100%」從來不是一個好消息。

圖 14-17　模型複雜度：那條 U 形曲線

14.6.2 偏差-變異權衡

這兩種失敗的背後，是一組深刻的張力：偏差-變異權衡(Bias-Variance Tradeoff)。偏差(Bias)是模型因為太簡單而產生的系統性誤差(對應擬合不足);變異(Variance)是模型因為太複雜、對訓練資料太敏感而產生的不穩定(對應過度擬合)。兩者往往此消彼長：提高模型複雜度會降低偏差、但增加變異;降低複雜度則反之。機器學習調校的藝術，很大程度就是在這兩者間尋找最佳平衡點——這也呼應了全書反覆出現的「取捨」主題。



上一張圖的 U 形曲線，可以再拆得更細。這裡換成迴歸問題（分類的分解在數學上沒那麼乾淨），做法是用同一個真實函數重複產生 200 組各 25 筆的小資料，每組各配一次多項式，再看這 200 條曲線的集體行為。偏差是「200 條的平均」離真實函數有多遠：1 次多項式（直線）再怎麼平均也追不上彎曲的真實函數，偏差² 高達 0.482，這是系統性的錯，再多資料也救不了。變異是「200 條彼此差多少」：9 次多項式每換一批資料就抖出完全不同的形狀，變異高達 34.59，是 1 次多項式的 634 倍;可是它的偏差² 只有 0.142，不到直線的三分之一——也就是說它平均起來並沒有偏掉，只是每一次都亂跑。這正是過度擬合的本質：不是學不會，是學得太隨資料而變。總誤差等於偏差² + 變異 + 雜訊，而雜訊 0.203 是資料本身帶的，任何模型都消不掉，這就是誤差的下限。兩者一降一升，總和在 5 次最低（0.303）。這個分解解釋了本章所有的旋鈕：kNN 的 k 變大、SVM 的 gamma 變小、樹變淺，都是在「用偏差換變異」;反之則是「用變異換偏差」。而加資料是唯一能單獨壓低變異、又不增加偏差的手段——這正是下一張圖第一件武器最強的理論原因。

圖 14-18　偏差與變異：把誤差拆成兩半

14.6.3 對抗過度擬合的武器

對抗過度擬合有幾件常備武器，它們在第 15 章訓練神經網路時會反覆出現：其一，更多資料——資料愈多，模型愈難靠死背過關，這再次呼應「資料是燃料」;其二，資料增強(Data Augmentation)——對訓練影像做旋轉、平移、翻轉、調亮度(全是第 4、5 章的技術!)人工擴充資料的多樣性，這是影像領域對抗過度擬合最有效的手段之一;其三，正則化(Regularization)——限制模型的複雜度，懲罰過大的參數，這與第 7 章影像還原的正則化是同一個數學思想(以約束對抗病態);其四，及早停止——在驗證表現開始變差時就停止訓練，不讓模型繼續往「背誦」的方向走。這四件武器常被並列，但它們的效果其實差了一個量級。圖 14-19 把四件都套在同一個模型與同一套訓練流程上實測(起點：訓練 100%、測試 88.2%)：更多真實資料 97.3%、資料增強 92.3%、正則化 89.2%、及早停止 88.7%，四件一起上 98.5%。成本的排序恰好相反——改參

數免費、寫增強程式便宜、收集標註最貴。因此實務上的建議順序是：正則化與及早停止先試，資料增強次之，收集真實資料最後，但要知道最後那一件才是效果最大的。另外要留意效果不會線性相加(它們壓的是同一個變異)，而且組合使用時，及早停止的輪數必須用該組合自己的驗證曲線重新找。

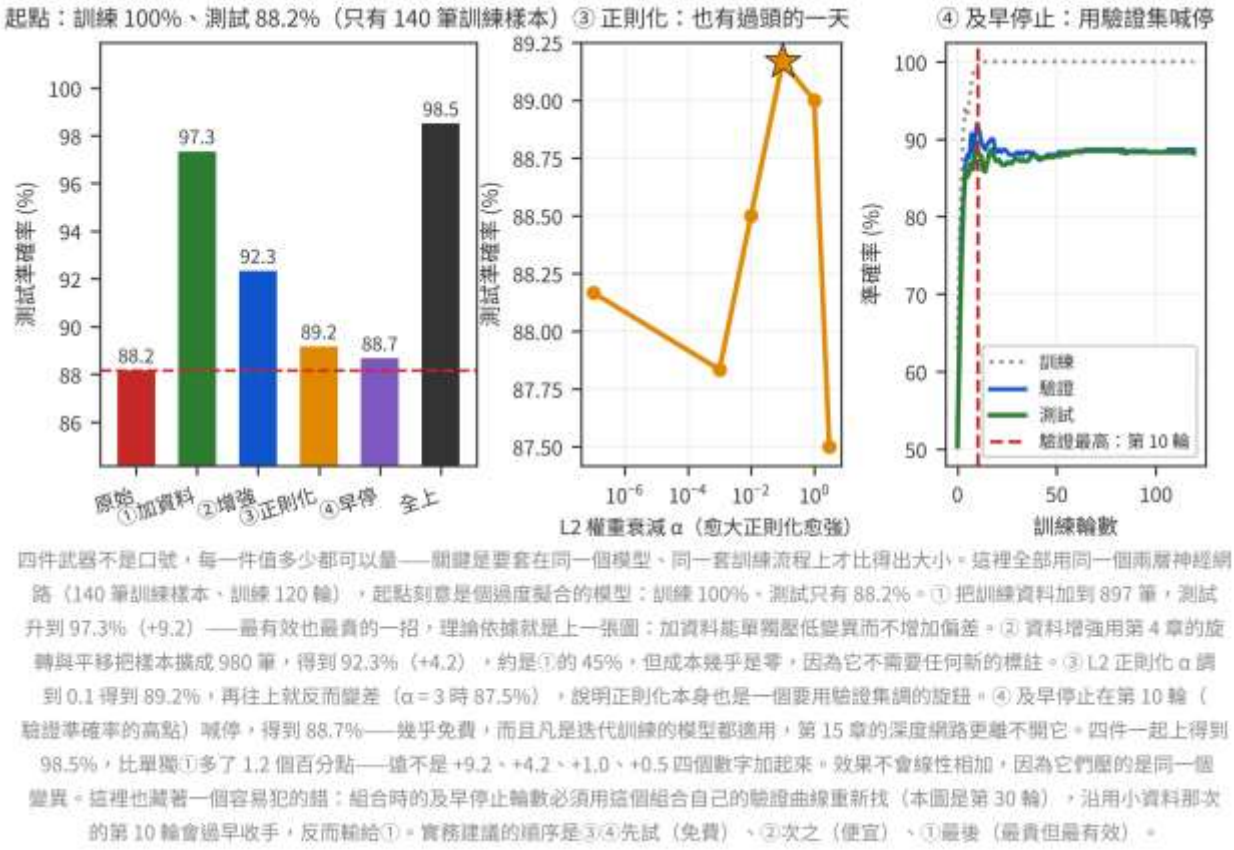


圖 14-19　四件武器，各值多少

請留意這裡的美妙連結：對抗過度擬合的資料增強，用的是第 4、5 章的幾何轉換與影像強化;正則化的思想來自第 7 章。機器學習不是憑空而來的新學問，它站在前面十三章的肩膀上——這正是本教科書把機器學習放在第 14 章、而非一開頭的用意。

14.6.4 推導：偏差-變異分解

14.6.2 節用文字說了偏差與變異此消彼長，圖 14-18 做了實測。這一小節把那條「總誤差 = 偏差² + 變異 + 雜訊」的等式推一遍。它只用到一個技巧——加一項再減一項——卻是整個機器學習理論裡最常被引用的分解。

為了推導乾淨，這裡用迴歸來談(分類問題的分解要麻煩得多，這也是圖 14-18 改用迴歸示範的原因)。設真實世界的關係是 y = f(x) + e，其中 f 是我們想學的真實函數、e 是量測雜訊，假設它平均為 0、變異數為 σ²，且與 x 無關。注意這個 σ² 是資料本身帶的，任何模型都消不掉——它是誤差的下限。

現在的關鍵是：我們的模型 g 是從一筆訓練資料學出來的，換一筆訓練資料就會學出不同的 g。所以 g(x) 是一個隨機變數，隨機性來自「抽到哪一筆訓練資料」。我們關心的是它在某個固定測試點 x 上的期望平方誤差 E[(y − g(x))²]，期望值同時對雜訊與訓練資料取。再令 m(x) = E[g(x)]，也就是「無限多次重抽訓練資料、學出無限多個模型之後的平均預測」——這正是圖 14-18 上排那條紅線的意思(以 200 條曲線的平均來近似)。

展開期望值，先把 y 換成 f(x) + e：

```
E[(y − g)²] = E[(f + e − g)²]
            = E[(f − g)²] + 2·E[e·(f − g)] + E[e²]
            = E[(f − g)²] + 0 + σ²
```

中間那項消失，是因為雜訊 e 與訓練資料無關、且平均為 0，所以 E[e·(f − g)] = E[e] · E[f − g] = 0。剩下的 E[(f − g)²] 就用那個加一項減一項的技巧：在括號裡同時加上與減去平均模型 m：

```
E[(f − g)²] = E[( (f − m) + (m − g) )²]
            = (f − m)² + 2·(f − m)·E[m − g] + E[(m − g)²]
            = (f − m)² + 0 + E[(g − m)²]
```

這一次中間項消失，是因為 m 的定義就是 g 的期望值，所以 E[m − g] = m − E[g] = 0;而 (f − m)² 不含隨機性，可以直接提到期望值之外。把兩段合起來，就得到了那條著名的等式：

```
E[(y − g(x))²]  =  (f(x) − m(x))²  +  E[(g(x) − m(x))²]  +  σ²
   總誤差       =       偏差²       +          變異         + 雜訊
```

三項各有清楚的意思。偏差² 是「平均模型離真實函數多遠」——它衡量模型族本身的表達能力不足，是系統性的錯，再多資料也救不了。變異是「每個模型離平均

模型多遠」——它衡量模型對訓練資料的敏感程度。雜訊則是資料自帶的下限。

圖 14-18 的實測把這三項都量了出來。1 次多項式(直線)配不上彎曲的真實函數，偏差² 高達 0.482、變異只有 0.055;9 次多項式每換一批資料就抖出完全不同的形狀，變異高達 34.59、是直線的 634 倍，但偏差² 只剩 0.142。兩者一降一升，總和在 5 次多項式處最低。而那個消不掉的雜訊項 $\sigma^2 = 0.45^2 = 0.203$，就是圖上那條怎麼樣都跨不過去的灰色虛線。

最後是這個分解的實務價值，它把本章所有的旋鈕串成了一條線。kNN 的 k 變大、SVM 的 gamma 變小、決策樹變淺、正則化係數變大——這些動作全都是「增加偏差、減少變異」;反方向則是「減少偏差、增加變異」。理解這一點，調參就不再是亂試：先診斷自己落在哪一端(訓練測試都差是偏差高、訓練好測試差是變異高)，再往反方向轉。而四件武器中「更多資料」之所以特別珍貴，正是因為它是唯一能單獨壓低變異、卻不增加偏差的手段——圖 14-19 實測它帶來 9.2 個百分點，是另外三件分別帶來的好幾倍。

14.7 應用案例

14.7.1 案例一：手寫數字辨識(傳統流程)

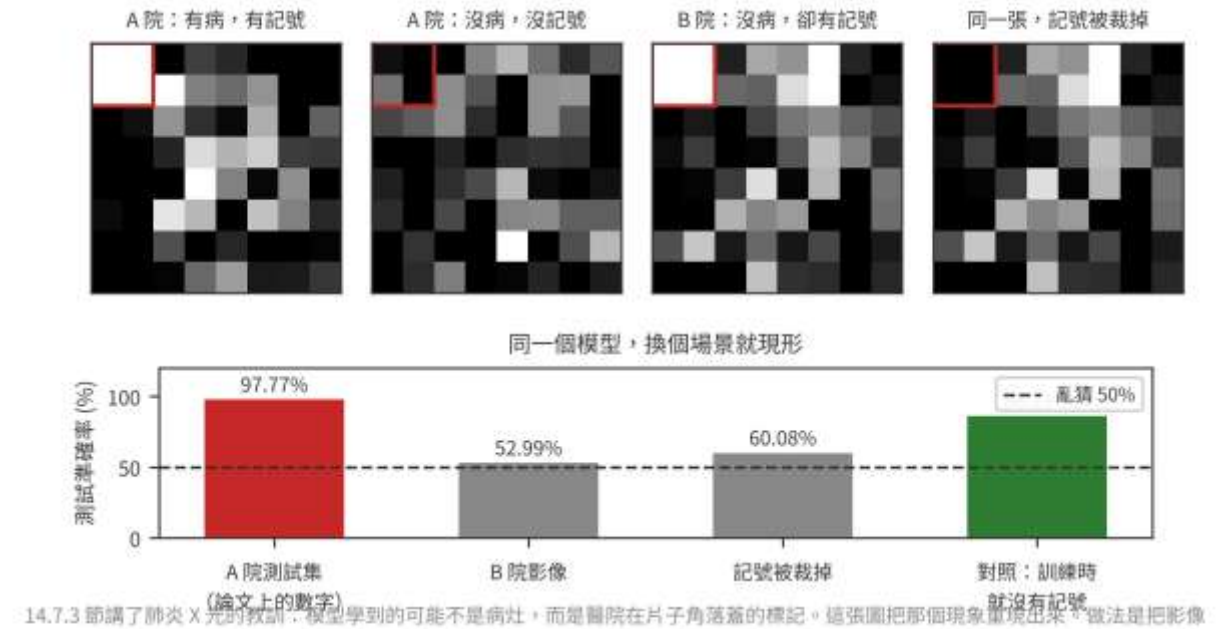
這是呼應叫獸開講、也是您課程 MNIST 實作的傳統版本。情境：辨識 0 到 9 的手寫數字，但不使用深度學習，以凸顯「特徵」這一環的重要。流程：對每張數字影像萃取特徵——可以是簡單的「像素攤平」，也可以是更聰明的 HOG(方向梯度直方圖，第 13 章特徵思想的延伸)→ 用 SVM 或 kNN 訓練分類器 → 以測試集評估、看混淆矩陣哪些數字最容易搞混(通常是 4 與 9、3 與 8、7 與 1)。這個案例讓學生親身體會：同樣的分類器，換用更好的特徵(HOG 優於原始像素)，準確率會顯著提升——這正是第 15 章深度學習要「自動學特徵」的動機所在。

14.7.2 案例二：水果或零件的品質分類

情境：農產分級(將水果依外觀分為特級、良級、次級)或工業零件的良品/不良品分類——這是機器學習在雲林在地農業與製造業最實際的應用。流程：拍攝大量已分級的樣本並標註 → 萃取特徵(第 8 章的顏色直方圖判斷成熟度、第 13 章的形狀特徵判斷外形、紋理特徵判斷表面瑕疵)→ 訓練 SVM 分類器 → 部署到分級生產線。這個案例的價值在於：它示範了前面各章的「特徵工程」如何餵給機器學習模型，是傳統影像處理與機器學習銜接的典型範例，也說明了為什麼在樣本有限的中小企業場景，「特徵工程 + SVM」常比動輒需要上萬標註樣本的深度學習更務實。

14.7.3 案例三：資料切分陷阱的真實教訓

這個案例不寫程式，而是講一個發人深省的故事，凸顯 14.5 節資料切分的重要。曾有研究團隊訓練模型從胸部 X 光辨識肺炎，在測試集上準確率極高，上線後卻表現崩壞。追查發現：訓練資料中，重症病人的 X 光多來自某幾家配備較舊機器的醫院，模型竟然「偷懶」學會了辨識影像角落的醫院標記與機器雜訊特徵，而非真正的病灶——它在資料中找到了一條與答案高度相關、卻毫無醫學意義的「捷徑」。因為測試集也來自同樣的醫院分布，這個捷徑在測試時依然有效，直到面對新醫院的資料才現形。



14.7.3 節講了肺炎 X 光的教訓：模型學到的可能不是病灶，而是醫院在片子角落蓋的標記。這張圖把那個現象重現出來：做法是把影像加重雜訊（讓真正的任務變難），再於左上角蓋一個與標籤有 97% 相關的小方塊，就像某家醫院的重症片則好都用同一台機器拍、帶著同一個記號。訓練與測試都來自這家醫院時，模型拿到 97.77%，論文、簡報、驗收報告都會很好看。但把同一個模型拿去 B 醫院（記號還在、只是與病情無關），準確率掉到 52.99%，幾乎就是丟銅板;把記號整個裁掉也只剩 60.08%——它從頭到尾幾乎沒在看影像本身。對照組最刺眼：訓練時就沒有記號的模型只能老實學影像，拿到 86.23%，比那個漂亮的 97.77% 低了 11.5 個百分點，卻是四根柱子裡唯一真的能用的模型。這件事最可怕的地方在於：如果測試集和訓練集來自同一個偏差分布，那麼再嚴謹的切分、再漂亮的交叉驗證都抓不到它——測試集本身就帶著同樣的捷徑。唯一的解法是換一批真正獨立來源的資料驗收，並且實際去看模型在看影像的哪裡（第 15 章的類別活化圖就是為此而生）。這是工程倫理，不只是技術細節。

圖 14-20　捷徑學習：測試分數很高，卻什麼都沒學到

這個真實教訓有兩層啟示：其一，資料的收集與切分若有偏差(不同類別來自不同來源)，模型會學到「捷徑」而非真正的規律，測試表現會給出虛假的信心。其二，這也呼應了第 7 章「還原 vs. 腦補」與第 9 章醫學影像的倫理提醒——在醫療等高風險領域，一個「準確率很高」的模型未必可信，必須深究它到底學到了什麼。機器學習工程師的責任，不只是把準確率調高，更是理解模型為何做出判斷、資料是否公平。這是技術素養，也是工程倫理。

14.8 本章與全書的觀念連結總表

相關章節	共用的觀念	機器學習中的角色
第 4 章 幾何轉換	旋轉、平移、翻轉	資料增強對抗過度擬合的主要手段
第 5 章 空間域	亮度、對比調整	資料增強的另一組操作

強化		
第 7 章 影像還原	正則化	對抗過度擬合的正則化與病態問題正則化同源
第 8 章 色彩處理	顏色直方圖	品質分類的顏色特徵
第 13 章 特徵擷取	SIFT、HOG、形狀特徵	傳統機器學習的「特徵」環節
第 15 章 深度學習	神經網路、自動特徵	深度學習是機器學習的分支，遵循同樣的訓練評估原則
第 16 章 物件偵測	COCO、mAP	COCO 資料集;評估指標延伸自本章
第 9、10 章 分割/二值	準確、召回、誤報漏檢	評估指標的取捨思維一以貫之

表 14-2 第 14 章與全書其他章節的觀念連結

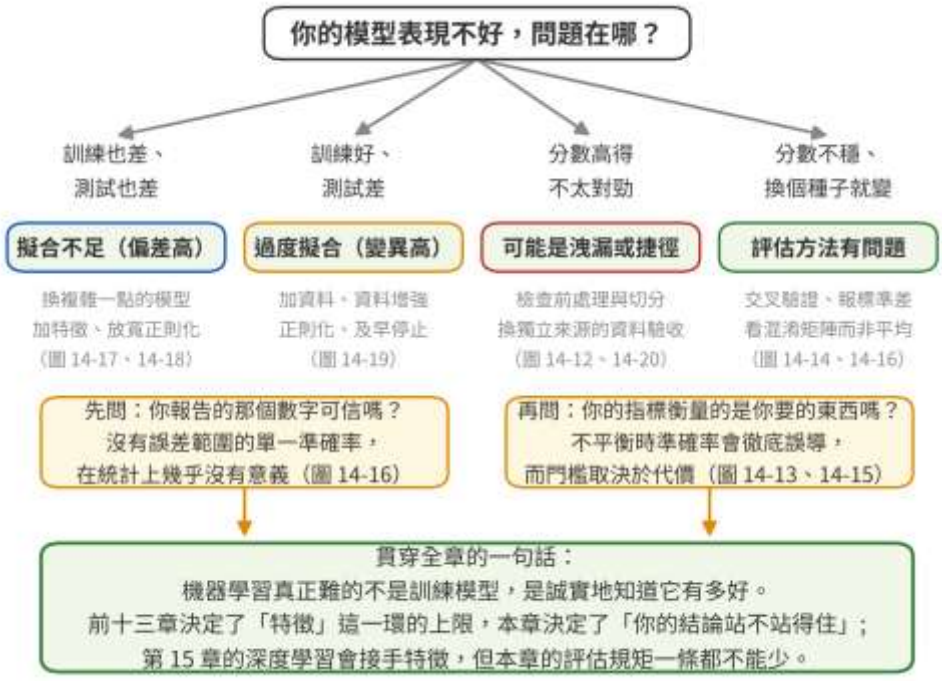


圖 14-21　第 14 章的決策地圖

程式實作 Lab 14

Lab 14-1 分類器擂台：kNN vs. SVM

目標：比較兩種經典分類器。(a)以程式 14-1 在手寫數字資料集上訓練 kNN 與 SVM，比較準確率與預測速度;(b)調整 kNN 的 k 值(1、5、15、50)，觀察準確率如何變化，找出最佳 k，並解釋 k 太小與太大各自的問題;(c)調整 SVM 的 C 與 gamma，觀察對結果的影響;(d)畫出兩者的混淆矩陣，找出最容易被搞混的數字對(如 4 與 9)，並思考為什麼;(e)討論：kNN 為什麼「不訓練」?這帶來什麼優點與缺點?

Lab 14-2 特徵的威力

目標：親身體會「特徵」這一環的決定性。(a)對同一個手寫數字資料集，分別用兩種特徵訓練 SVM：一是「原始像素攤平」，二是「HOG 特徵」;(b)比較兩者的準確率——通常 HOG 會明顯勝出;(c)思考：為什麼更好的特徵能提升同一個分類器的表現?這說明了傳統機器學習中「特徵工程」的重要性;(d)延伸討論：第 15 章的深度學習如何「自動」完成這件事，免去人工設計特徵的負擔?

Lab 14-3 過度擬合的現形與對抗

目標：親眼見證過度擬合。(a)刻意用一個很複雜的模型(如 SVM 的 gamma 設得很大)在少量訓練資料上訓練，記錄訓練準確率(會很高)與測試準確率(會很低)，這就是過度擬合;(b)畫出「訓練集大小 vs. 訓練/測試準確率」的學習曲線，觀察兩條線的落差如何隨資料增加而縮小——驗證「更多資料對抗過度擬合」;(c)加入資料增強(用第 4 章旋轉、第 5 章調亮度擴充訓練集)，觀察測試準確率是否提升;(d)討論：如何從學習曲線判斷一個模型是擬合不足還是過度擬合?各該怎麼補救?

Lab 14-4 品質分類器(整合前面各章)

目標：打造一個結合影像處理與機器學習的實用系統。(a)拍攝或收集兩三類物件的影像(如不同成熟度的水果、良品與不良品的零件)，各數十張並標註;(b)萃取特徵：

用第 8 章顏色直方圖 + 第 13 章形狀/紋理特徵，組成特徵向量;(c)正確切分訓練/測試集，訓練 SVM 分類器;(d)以混淆矩陣、精確率、召回率評估，並依應用場景(如「不良品絕不能放過」)判斷該重召回還是重精確;(e)反思：你的資料收集有沒有落入 14.7.3 節的「捷徑陷阱」?(例如所有良品都在同一背景下拍攝，模型會不會學到背景而非物件?)

Lab 14-5 門檻、代價與不平衡：把商業判斷寫進模型

目標：體會準確率的陷阱，並親手把代價結構轉成判定門檻。(a)用 load_digits 造一個人工的不平衡問題：挑一個數字當「不良品」，只保留十幾筆，其餘全當良品，計算不良品的佔比;(b)訓練一個未做任何處理的 SVM，記錄準確率、精確率、召回率與 F1——你應該會看到很高的準確率搭配 0% 的召回率，並與「一律判良品」這個什麼都沒學的基準比較，確認兩者分數相同;(c)加上 class_weight="balanced" 重訓，觀察四個指標各自怎麼變;(d)改用 predict_proba 取得分數，把門檻從 0.02 掃到 0.98，畫出精確率、召回率、F1 隨門檻變化的三條曲線，找出 F1 最高的門檻，並與預設的 0.5 比較差了幾個百分點;(e)設定三種代價(漏抓 10 倍貴、一樣貴、誤報 10 倍貴)，各自算出總代價隨門檻的曲線與最佳門檻，確認三者不同;(f)寫一段 150 字的說明，向一位不懂機器學習的廠長解釋「為什麼我們不用準確率驗收，以及為什麼門檻要由您來決定」。延伸：改用 average_precision_score 計算 AP，說明它為什麼與門檻無關。

Lab 14-6 誠實評估工作坊：切分、洩漏與捷徑

目標：親手製造三種常見的評估錯誤，再親手修好，把「誠實地知道模型有多好」變成肌肉記憶。(a)單次切分的不穩：固定資料與模型，只換 40 個隨機種子做 7:3 切分，記錄 40 個測試準確率，畫出直方圖，算出全距與標準差;再對同一份資料做 40 次 5 折交叉驗證取平均，比較兩組的標準差差幾倍;(b)排名翻轉：在同樣的 40 次切分中同時評估 SVM 與 kNN，統計「誰比較好」翻轉了幾次，寫下你對「只切一次就選模型」的結論;(c)製造洩漏：取一小批資料，把標籤完全打亂(此時真實準確率上限等於

亂猜)，然後故意在切分前用全部資料做特徵選擇，看測試準確率能虛高到多少;再改成先切分、只在訓練集上做特徵選擇，確認分數掉回亂猜水準;(d) 用 sklearn.pipeline.Pipeline 把前處理與模型包起來重做一次(c)，說明為什麼 Pipeline 從結構上就防止了這個錯誤;(e)製造捷徑：在每張影像的角落蓋一個與標籤高度相關的小方塊，訓練後分別在「同分布測試集」、「記號與標籤無關的影像」、「記號被裁掉的影像」三種情境下評估，並與「訓練時就沒有記號」的對照模型比較;(f)把上述所有數字整理成一頁報告，每個數字都附上標準差或信賴區間。延伸：用 sklearn.model_selection.GroupKFold 示範「同一來源的樣本不可跨越切分」，說明它在(e)那種場景能擋下什麼。

本章重點整理

- 機器學習是典範轉移：從「人寫規則」到「給範例讓機器歸納規則」;分監督式(有標籤，含分類與迴歸)、非監督式(無標籤，分群/降維)、強化學習(獎懲互動)三類，本章與深度學習聚焦監督式影像分類。
- 影像分類五環節：資料、特徵、模型、訓練、評估;傳統機器學習依賴人工設計特徵(第 13 章)，深度學習則自動學特徵——這是兩者的分野。
- 重要資料集：MNIST(入門 Hello World)、CIFAR(彩色自然物體)、ImageNet/ILSVRC(2012 引爆深度學習)、COCO(複雜場景偵測分割);資料是機器學習的燃料，常比演算法更決定成敗。
- kNN 以最近 k 個鄰居投票、不訓練但預測慢;SVM 以最大間隔超平面分類，核技巧處理非線性，是深度學習前的黃金分類器。
- 資料須切分訓練/驗證/測試，測試集嚴格保密以模擬真實表現;準確率在類別不平衡時會誤導，須輔以混淆矩陣、精確率(不誤報)、召回率(不漏抓)，依應用代價權衡。

- 擬合不足(太簡單、訓練測試都差)、過度擬合(太複雜、背下雜訊、訓練好測試差)源於偏差-變異權衡;對抗過度擬合靠更多資料、資料增強(第 4、5 章)、正則化(第 7 章)、及早停止。

習題

1. 機器學習相對於傳統程式設計，發生了什麼「典範轉移」?為什麼有些問題非用機器學習不可?

   提示與參考解答：傳統程式：人寫死規則，電腦套用到資料得答案。機器學習：人給資料和答案，電腦自己歸納出規則。轉移在於「從寫規則到給範例」。有些問題(辨認手寫數字、人臉、語音)的規則複雜到人類根本寫不出來，但範例俯拾皆是，只能讓機器從範例自己學。

2. 請說明監督式、非監督式、強化學習的差別，各舉一個影像或機器人相關的例子。

   提示與參考解答：監督式：資料有輸入與正確標籤，學習預測答案(如 MNIST 數字分類)。非監督式：資料無標籤，自己發現結構(如把相似影像自動分群)。強化學習：智能體與環境互動、依獎懲學策略(如機器人學會抓取物件、AlphaGo 下圍棋)。

3. 影像分類的五個環節是什麼?傳統機器學習與深度學習在哪一個環節產生了根本差異?

   提示與參考解答：五環節：資料、特徵、模型、訓練、評估。根本差異在「特徵」：傳統機器學習依賴人工設計特徵(SIFT、HOG)，模型只做「特徵到類別」;深度學習把「像素到特徵」與「特徵到類別」整合進同一網路，端到端自動學習特徵。

4. 為什麼說「資料是機器學習的燃料」?請以 ImageNet 對深度學習的影響說明。

   提示與參考解答：演算法是引擎、資料是燃料，沒燃料引擎跑不動。ImageNet(李飛飛團隊，逾 1400 萬張影像)提供了規模空前的訓練資料，2012 年 AlexNet 正是靠它在 ILSVRC 奪冠、引爆深度學習革命。沒有 ImageNet 就沒有深度學習爆發—

—是「先建大資料集」的洞見為演算法鋪好跑道，印證資料常比演算法更重要。

5. 承叫獸開講：MNIST 為什麼要混合「人口普查局員工」與「高中生」兩種來源的手寫數字?這與資料切分有什麼關聯?

   提示與參考解答：普查局員工字跡工整(較易)、高中生字跡潦草(較難)，混合是為了讓訓練集與測試集的難度分布一致，結論才可靠。若訓練全易、測試全難會低估模型能力，反之則高估。這正是資料切分公平性的體現——訓練與測試的分布必須一致，評估才可信。

6. kNN 的分類原理是什麼?為什麼說它「不訓練」?它有哪些缺點?

   提示與參考解答：原理：看新樣本在特徵空間最近的 k 個鄰居，由它們投票決定類別(物以類聚)。「不訓練」是因為它沒有需學習調整的參數，只把訓練樣本記住，預測時才算距離找鄰居。缺點：預測慢(每次都要比全部樣本)、記憶體耗費大、對特徵尺度與高維敏感(維度災難)。

7. SVM 的「最大間隔」原則是什麼?為什麼它有助於泛化?什麼是核技巧?

   提示與參考解答：最大間隔：不只要能分開兩類，還要讓分界線(超平面)離兩邊最近樣本盡可能遠;決定分界線的關鍵樣本即支持向量。它有助泛化，因為留有餘裕、對新樣本更有容錯能力。核技巧：當資料在原空間線性不可分時，隱含地映射到更高維空間使其線性可分，且不必真的計算高維座標，讓 SVM 能處理非線性分類。

8. 為什麼評估模型不能用訓練時看過的資料?訓練、驗證、測試三個集合各扮演什麼角色?

   提示與參考解答：用看過的資料評估，無法分辨模型是「學會規律」還是只「背下答案」(如考背過的題)。訓練集調整參數;驗證集調超參數、選模型、決定何時停止;測試集在一切定案後最後評估，模擬上線遇到全新資料的表現，須嚴格保密不可反覆使用，否則評估會過度樂觀。

9. 準確率在什麼情況下會誤導?請以罕見疾病為例，並說明精確率與召回率如何補救。

   提示與參考解答：類別不平衡時誤導：某病盛行率 1%，「一律判沒病」的模型準確率 99% 卻抓不出任何病人。精確率(判為陽性中真陽性的比例，衡量不誤報)與召回率(真陽性中被抓出的比例，衡量不漏抓)能揭示這個問題;癌症篩檢重召回(漏抓代價

大)，垃圾郵件重精確(誤判重要郵件代價大)。

10. 過度擬合與擬合不足各是什麼?如何從訓練與驗證表現診斷?它們與偏差-變異權衡有何關係?

    提示與參考解答：擬合不足：模型太簡單、抓不住規律，訓練與測試都差(高偏差)。過度擬合：模型太複雜、背下雜訊，訓練好但測試崩壞(高變異)。診斷：兩者都差是擬合不足;訓練好、驗證差且落差大是過度擬合。偏差-變異權衡：提高複雜度降偏差但增變異，反之亦然，調校即在兩者間找平衡。

11. 對抗過度擬合有哪四種常備武器?其中資料增強與正則化分別與本書哪幾章相關?

    提示與參考解答：四武器：更多資料、資料增強、正則化、及早停止。資料增強用第 4 章幾何轉換(旋轉、平移、翻轉)與第 5 章影像強化(調亮度、對比)擴充訓練資料多樣性;正則化(限制模型複雜度、懲罰過大參數)與第 7 章影像還原的正則化同源——都是以約束對抗病態/過度擬合。

12. (綜合思辨題)承 14.7.3 節肺炎 X 光的教訓：一個測試準確率很高的模型，為什麼上線後可能崩壞?這給機器學習工程師什麼責任上的啟示?

    提示與參考解答：模型可能學到與答案高度相關、卻無實質意義的「捷徑」(如醫院標記、機器雜訊)，而非真正的病灶。若測試集與訓練集來自同樣的偏差分布，捷徑在測試時仍有效，直到面對新分布才現形。啟示：資料收集與切分若有偏差，模型會學捷徑、測試給出虛假信心;工程師的責任不只是調高準確率，更要理解模型為何做出判斷、資料是否公平——這是技術素養，也是工程倫理(呼應第 7、9 章)。

13. 為什麼「訓練準確率 100%」通常不是一個好消息?請用圖 14-17 的兩條曲線說明，並說明該用什麼來決定模型複雜度。

    提示與參考解答：訓練準確率隨複雜度單調上升，模型複雜到一定程度就會把訓練資料(連同其中的錯誤標註與雜訊)背下來，因此 100% 只代表「背完了」，不代表學會規律。圖 14-17 實測：決策樹深度加到 20 以上時訓練 100%，測試卻從深度 8 的 68.5% 掉到 61.0%;SVM 的 gamma 加大時訓練 100%，測試從 92.8% 崩到 12.0%(十類問題下幾乎等於亂猜)。兩條線之間的落差就是過度擬合的量。決定複雜度只能靠驗證集(或交叉驗證)，因為訓練準確率在過度擬合區是一路上升的，完全沒有提供「該停在哪裡」的訊號。

14. 手上只有 200 筆測試樣本，你的模型測得 90% 準確率。請用 14.5.3 節的公式算出 95% 信賴區間，並說明若另一個模型測得 92%，你能不能宣稱它比較好。

    提示與參考解答：SE = √(0.9 × 0.1 / 200) = √0.00045 ≈ 0.0212，即 2.12 個百分點;95% 區間為 90% ± 1.96 × 2.12% ≈ 90% ± 4.2%，也就是約 85.8% 到 94.2%。92% 落在這個區間內，兩個區間重疊得很厲害，因此不能宣稱較好——這個差距在統計上無法與抽樣雜訊區分。要做出可信的宣稱，有三條路：把測試集加大(SE 隨 √n 下降，要把誤差砍半需四倍樣本)、改用交叉驗證並報告標準差、或在多個隨機種子下重複實驗看排名是否穩定。圖 14-16 的實測提醒：同一份資料只換種子，單次切分的結果就能差 8.9 個百分點。

15. 什麼是「資料洩漏」(data leakage)?請舉出三種常見的洩漏途徑，並說明為什麼洩漏比單純的過度擬合更危險。

    提示與參考解答：資料洩漏指訓練過程中以任何形式用到了測試集的資訊，使測試分數不再代表真實表現。三種常見途徑：(1)前處理先做再切分——標準化、特徵選擇、PCA 若在全部資料上擬合，測試集的統計量就已經流進模型;(2)資料增強先做再切分——同一張原圖的增強版本同時出現在訓練與測試集，等於拿看過的東西考試;(3)反覆用測試集調參——每看一次測試集做一次決定，就洩漏一點點，累積起來就讓測試集失效。洩漏比過度擬合更危險，是因為過度擬合會被誠實的測試集抓到，洩漏卻讓測試集本身失去偵測能力——分數漂亮、問題看不見，直到上線才爆發。圖 14-12 的實測顯示：在完全隨機的標籤上(真實準確率上限 50%)，先做特徵選擇再切分能讓測試準確率虛高到 75%。

16. 承上題：請寫出正確的前處理順序，並說明 scikit-learn 的 Pipeline 為什麼能從結構上防止這類錯誤。

    提示與參考解答：正確順序是「先切分 → 只在訓練集上 fit 前處理器 → 用同一個前處理器 transform 訓練集與測試集」。關鍵是測試集只能被 transform、絕不能參與 fit。Pipeline 把前處理與模型包成一個物件，呼叫 pipeline.fit(X_tr, y_tr) 時，它只會在訓練資料上擬合每一個前處理步驟;在交叉驗證中(cross_val_score(pipeline, X, y, cv=5))，每一折都會重新擬合一次前處理器，確保該折的驗證資料完全沒參與。若不用 Pipeline、自己先做 scaler.fit(X) 再切分，就犯了上題的第一種洩漏。所以 Pipeline 不只是寫法簡潔，它是一種把正確流程寫死在結構裡的防呆設計。

17. 在一條瑕疵檢測產線上，不良品只佔 0.86%。請說明為什麼「一律判良品」的模型能拿到 99.12% 的準確率，以及你該改用哪些指標，又該如何讓模型真的抓到不良品。

    提示與參考解答：準確率只計算「判對的比例」，當某一類佔了 99% 以上，全部判成那一類就自動拿到 99% 以上。圖 14-13 的實測：盛行率 0.86% 時，「一律說良品」的準確率 99.12%，但精確率與召回率都是 0——它一個不良品也沒抓到，對產線毫無價值。更警醒的是，未做處理的 SVM 給出一模一樣的 99.12% 與 0% 召回率，因為最小化整體錯誤的最快方法就是放棄少數類。應改看混淆矩陣、精確率、召回率、F1，以及與門檻無關的 AP(圖 14-15)。讓模型真的抓到不良品的手段包括：設定類別權重(實測 class_weight 讓召回率從 0% 升到 80%、準確率 99.82%)、重抽樣(過採樣少數類或欠採樣多數類)、調整判定門檻、以及在評估時改用對少數類敏感的指標。

18. 為什麼說「0.5 從來不是一個神聖的門檻」?請以漏抓代價是誤報十倍、與誤報代價是漏抓十倍兩種情境，說明最佳門檻會往哪個方向移動，以及這個決定該由誰來做。

    提示與參考解答：分類器輸出的是分數或機率，0.5 只是預設的切點，與任務的代價結構毫無關係。門檻調低會判更多樣本為陽性，召回率上升、精確率下降(寧可錯殺);門檻調高則反之(寧可放過)。漏抓十倍貴時(例如瑕疵品流到客戶手上)，最佳門檻往下移，寧可多攔一些;誤報十倍貴時(例如誤判害整批重工)，最佳門檻往上移。圖 14-15 的實測：同一個模型、同一批資料，三種代價結構下的最佳門檻分別是 0.04、0.60 與 0.90。這個決定不是演算法能自動算出來的，必須由懂現場的人提供兩類錯誤的相對代價——它是把商業判斷寫進系統的那一步，也呼應第 9 章 Canny 雙門檻與第 10 章 Viola-Jones 的同一類思維。

19. 請說明交叉驗證的做法，以及它解決了單次切分的什麼問題。當資料量達到數十萬筆時，還需要交叉驗證嗎?

    提示與參考解答：k 折交叉驗證把資料切成 k 等份，輪流拿一份當驗證集、其餘 k−1 份訓練，共做 k 次，每一筆樣本都剛好被驗證到一次，最後取平均。它解決的是單次切分的高變異：圖 14-16 實測同一份 300 筆資料、只換 40 個隨機種子，單次切分的結果從 90.0% 到 98.9%、標準差 2.14;5 折平均的標準差降到 0.66，穩定

約 3.3 倍。更嚴重的是排名會翻轉——該實驗中 SVM 與 kNN 的勝負在 40 次裡翻轉了 13 次。資料達數十萬筆時，單一次切分出來的測試集本身就夠大(由 14.5.3 節的公式，n = 100,000 時標準誤只有 0.1% 上下)，加上交叉驗證要把訓練成本乘以 k 倍，因此大資料場景通常改用單次固定切分;但分層抽樣、以及「測試集只在最後用一次」的紀律依然不能省。

20. 請用偏差-變異的語言，說明以下四個調整各自往哪個方向移動：(a)kNN 的 k 從 1 調到 50;(b)SVM 的 gamma 加大;(c)決策樹的最大深度變淺;(d)訓練資料從 140 筆加到 897 筆。

    提示與參考解答：(a)k 變大 → 更多鄰居投票、決策邊界變平滑 → 變異下降、偏差上升(k=1 時訓練準確率 100% 是典型的高變異)。(b)gamma 加大 → 每個支持向量影響範圍變小、邊界變皺 → 變異上升、偏差下降(圖 14-17 實測 gamma 過大時測試掉到 12.0%)。(c)樹變淺 → 模型變簡單 → 變異下降、偏差上升。(d)加資料是特例：它單獨壓低變異而不增加偏差，因為模型族沒有改變、只是估計得更穩(圖 14-19 實測 +9.2 個百分點，遠勝其他三件武器)。前三項都是在同一條權衡曲線上左右移動，只有(d)是把整條曲線往下推——這正是「資料是燃料」這句話的理論版本。

21. 對抗過度擬合的四件武器，實測效果分別是多少?請依「成本」與「效果」排出你的實務使用順序，並說明理由。

    提示與參考解答：圖 14-19 在同一個模型與訓練流程上量過四件武器(起點：140 筆訓練樣本，訓練 100%、測試 88.2%)：更多真實資料 97.3%(+9.2)、資料增強 92.3%(+4.2)、正則化 89.2%(+1.0)、及早停止 88.7%(+0.5)，四件一起上 98.5%。成本排序正好相反：正則化與及早停止只是改參數，幾乎零成本;資料增強只需程式(第 4、5 章的旋轉、平移、亮度調整)，不必新標註;收集並標註真實資料最貴。因此實務順序建議是先試正則化與及早停止、再做資料增強、最後才投資收集資料。要注意效果不會線性相加(它們壓的是同一個變異)，而且組合使用時及早停止的輪數必須用該組合自己的驗證曲線重新找——沿用小資料時的輪數會過早收手。

22. 什麼是「捷徑學習」(shortcut learning)?為什麼嚴謹的資料切分與交叉驗證都抓不到它?該怎麼辦?

    提示與參考解答：捷徑學習指模型找到一個與答案高度相關、卻與任務本質無關的特徵(如醫院在片子角落蓋的標記、不同來源的機器雜訊、背景或拍攝條件的差異)，

並用它取代真正該學的規律。切分與交叉驗證抓不到它，是因為這兩者都假設訓練集與測試集來自同一個分布;若偏差存在於整份資料中，測試集就帶著同樣的捷徑，分數依然漂亮。圖 14-20 的實測：加上與標籤 97% 相關的角落記號後，同分布測試集拿到 97.77%，換一家「醫院」掉到 52.99%、把記號裁掉只剩 60.08%(二元任務亂猜是 50%);而訓練時就沒有記號的對照模型只有 86.23%，卻是唯一能用的。對策：用真正獨立來源的資料驗收、刻意做跨來源(跨醫院、跨產線、跨季節)測試、用可解釋性工具(第 15 章的類別活化圖)實際去看模型在看哪裡、並在資料收集階段就避免讓類別與來源綁在一起。

23. (推導題)承 14.4.3 節：請說明為什麼可以自由把 w 與 b 同乘一個常數，以及 SVM 為什麼要把最近樣本的 |w·x + b| 訂為 1。若改訂為 2，最後的目標函數會變成什麼?

    提示與參考解答：$w·x + b = 0$ 與 $(cw)·x + cb = 0$ 對任何 $c \neq 0$ 描述的是同一個超平面，因為把方程式兩邊同乘常數不改變解集合。這表示 (w, b) 有一個多餘的縮放自由度，可以任意訂定而不失一般性。訂為 1 是為了讓約束寫成乾淨的 $y·(w·x + b) \geq 1$，並讓間隔有簡潔的表達式 $2 / |w|$。若改訂為 2，則最近樣本滿足 $|w·x + b| = 2$，距離變成 $2 / |w|$、間隔變成 $4 / |w|$，約束變成 $y·(w·x + b) \geq 2$，目標仍是最小化 $|w|$(只差一個常數倍)。換言之，最佳的那個超平面完全沒變，變的只是 w 的長度單位——這正說明了那個 1 是一個方便的約定，不是數學上的必要條件。

24. (推導題)承 14.6.4 節：在偏差-變異分解中，為什麼交叉項 E[e·(f – g)] 與 E[m – g] 都會消失?若雜訊 e 的平均不是 0 而是某個常數 c，等式會變成什麼?

    提示與參考解答：第一個交叉項消失，是因為雜訊 e 與訓練資料獨立，所以期望值可以拆開成 $E[e] · E[f – g]$，而 $E[e] = 0$。第二個交叉項消失，是因為 m 依定義就是 g 的期望值，所以 $E[m – g] = m – E[g] = m – m = 0$。兩者都用到「某一項的期望為 0」這個事實，但理由不同：前者來自雜訊的假設，後者來自平均模型的定義。若 $E[e] = c \neq 0$，則 $E[e^2] = \sigma^2 + c^2$，且交叉項變成 $2c · E[f – g] = 2c · (f – m)$，等式多出兩項。實務上這表示量測系統有系統性偏移(例如感測器未歸零)，它會被模型吸收成一個常數偏差;正確的做法是先校正資料(第 5 章的影像強化與第 7 章的還原)，而不是讓模型去補償。

25. (綜合實作思辨題)你要為雲林一家柑橘包裝廠做一套分級系統，目前只拿到 300 張已分級的照片，全部在同一個上午、同一條產線拍攝。請規劃完整流程，並指出這份資料最可能埋下哪三個陷阱、各要如何處置。

    提示與參考解答：流程應涵蓋本章五環節——資料(切分策略先訂好)、特徵(第 8 章顏色直方圖判成熟度、第 13 章 HOG/LBP 判外形與表面)、模型(樣本少，SVM 優於深度學習)、訓練(交叉驗證調 C 與 gamma)、評估(混淆矩陣、各級的精確率與召回率、依代價選門檻)。三個陷阱：(1)資料量太少——300 張分三級、再切測試集後每級只剩幾十張，由 14.5.3 節的公式，測試集 60 筆時標準誤高達 ±6 個百分點，任何小幅改善都不可信;處置是用交叉驗證、報標準差，並持續累積資料。(2)捷徑與分布偏差——同一個上午、同一條產線代表光照、背景、相機角度完全一致，模型極可能學到與分級相關的拍攝條件而非果實本身(圖 14-20 的教訓);處置是跨時段、跨天候、跨批次重新收集驗收資料，並做資料增強模擬光照變化(第 5 章)。(3)類別不平衡——次級品通常遠少於良品，準確率會被多數類灌水(圖 14-13);處置是看混淆矩陣與各級召回率、設類別權重、並與廠方確認「漏判次級品」與「誤判良品」的實際代價，據此定門檻(圖 14-15)。最後要留一句工程倫理：上線後仍須持續抽檢回饋，因為柑橘的外觀會隨季節與產區改變，模型會隨時間漂移。

參考資料與延伸閱讀

- Gonzalez, R. C., & Woods, R. E., Digital Image Processing, 4th ed., Pearson, 2018, Ch. 12(物件辨識與分類基礎)。
- LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P., Gradient-Based Learning Applied to Document Recognition, Proc. IEEE, 1998.(LeNet 與 MNIST，本章叫獸開講之史料來源)
- Deng, J., et al. (Li, F.-F.), ImageNet: A Large-Scale Hierarchical Image Database, CVPR, 2009.
- Cortes, C., & Vapnik, V., Support-Vector Networks, Machine Learning, 1995.(SVM 原始論文)

- Lin, T.-Y., et al., Microsoft COCO: Common Objects in Context, ECCV, 2014.
- Bishop, C. M., Pattern Recognition and Machine Learning, Springer, 2006.(機器學習經典教科書)
- 補充教材與課程資訊：@prof(https://page.line.me/prof)